ZipperGen Workflow Development and Deployment Manual

Concepts, commands, configuration, runtime, and operations

The ZipperGen Authors

Manual edition: 28 July 2026

Abstract

This manual is the comprehensive reference for developing and operating ZipperGen applications. It explains the Studio lifecycle, global workflows and local projections, specifications and coding-assistant handoffs, model configuration, semantic inspection, durable execution, human approval, deployment, supervision, recovery, troubleshooting, and the corresponding scriptable commands. It is written for readers who can use a terminal and editor but do not need to be experienced Python or operations engineers.

Looking for the shortest path?

The companion `docs/first-workflow.tex` is the self-contained first-hour tutorial. It goes directly from a new directory to a specified, inspected, mock-run, resumed, and prepared deployment. Start there when building a first workflow; return to this manual when a step needs more explanation or when configuring real models and production operations.

The main promise

The workflow is always ordinary Python code. A coding assistant may create or change that code from one visible, versionable workflow specification, but ZipperGen performs the deterministic work: loading it, validating every projection, showing what each participant does, comparing semantics, storing durable execution state, and managing deployment. LLMs propose workflows; ZipperGen makes them explicit, verifiable, and deliberately deployable. It never treats opaque model output as an automatically deployed workflow.

Contents

1 The mental model

1.1 A workflow is a global protocol

Most agent frameworks distribute control logic among agents. ZipperGen instead starts with one global protocol: who owns a value, who sends it, who receives it, who performs an action, and who owns each decision. From that global code, ZipperGen projects an exact local program for every participant.

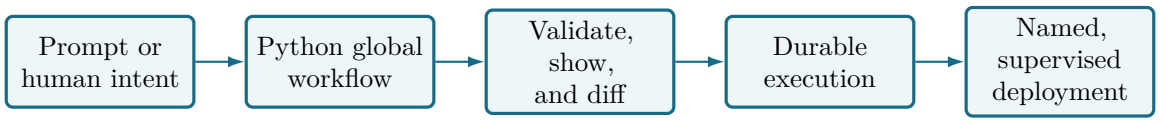

Figure 1: The prompt-to-production path. The Python workflow remains the reviewable source of truth in the middle of the cycle.

For a well-formed protocol, every projected send has a corresponding receive. The code therefore serves simultaneously as executable behavior, design documentation, and an audit surface.

A global protocol does not have to be one monolithic function. Reusable or conceptually coherent subsequences can be given meaningful names so that the top-level protocol stays a reasonable size. Decorate such a helper with `@fragment`, then call it inside a `@workflow` body:

```
from zippergen import fragment, workflow

@fragment
def request_review(draft):
    Writer(draft) >> Reviewer(draft)
    Reviewer: approved = review(draft)
    Reviewer(approved) >> Writer(approved)

@workflow
def reviewed_answer(request: str @ Requester):
    Requester(request) >> Writer(request)
    Writer: draft = write_draft(request)
    request_review(draft)
```

A fragment is an authoring-time, reusable piece of the same global protocol. Its statements are inlined into the surrounding workflow before validation and projection; it is not a separately deployed workflow, participant, process, or durable run. It can still be useful when called only once if it names a coherent protocol stage. Use another top-level `@workflow` only when the unit should be loaded, run, and deployed independently.

1.2 Important words

Term	Meaning
Lifeline	One sequential participant or responsibility boundary. It may represent an agent, a person, a mailbox, or an external-system role.
Message	An explicit transfer of one or more values from one lifeline to another, written with `Sender(value) >> Receiver(value)`.
Action	Work performed locally at one lifeline: deterministic computation, LLM work, an external effect, human input, or runtime planning.

Term	Meaning
Owned decision	An `if` or `while` guard followed by `@ Lifeline`. That lifeline must know the guard value and owns the choice.
Projection	The local program generated for one lifeline from the global protocol.
Workflow spec	A CLI reference such as `zippergen/examples/tutorial_review.py:tutorial_review`: file path, colon, workflow object name. It selects an existing workflow object; it does not name a deployment.
Store	A SQLite file holding messages, actions, traces, human tasks, and workflow results. It enables durable resume and replay.
Human task	A durable pending request created by a `@human` action. It contains the question or rendered content, its pending/completed state, and the eventual human decision.
Task ID	The stable internal identifier of a human task. A trusted local operator may use it with `approve --task`. It identifies the task but is not intended to be an approval credential in an untrusted external link.
Task token	A durable random bearer value that maps back to one human task. External adapters can use it with `approve --token`; treat it like a secret approval link. A token does not create another task.
Token channel	An arbitrary label such as `email`, `telegram`, or `cli`. It allows the same task to have a separate token for each adapter; it is not a workflow message channel.
Deployment profile	Persistent non-secret settings for a named deployment.
Bundle	A timestamped copy of the workflow and its declared support files. Deployments run from this snapshot, not from a file that may change halfway through execution.
Managed environment	A deployment-specific Python virtual environment with ZipperGen and declared packages installed.
Service manager	macOS `launchd` or Linux user `systemd`, used to start, restart, and supervise a deployment.

1.3 Action kinds

Decorator	Use it for	Important property
`@pure`	Deterministic local computation	No external I/O or durable side effect.
`@effect`	Email, files, APIs, databases, or other external mutation	Make it retry-safe or idempotent.
`@assistant`	Repository-aware work performed by Codex, Claude Code, or a custom assistant backend	The action, Markdown instruction fingerprint, typed inputs, owner, and result remain inspectable and durable.
`@llm`	Model generation or judgment	Declare prompts, parse format, and typed outputs.
`@human`	Confirmation, edit, selection, acknowledgement, or input	Creates a durable task in SQLite execution.
`@planner`	A model-generated allowed subworkflow at runtime	Uses a fail-closed statement grammar. Generated imports, Python bodies, and `@pure` helpers are rejected before import.

A planner may generate schema-checked `@llm` actions when its `allow` declaration includes `llm`. Deterministic `@pure` helpers must be defined in reviewed source and passed through the planner's

`actions` argument. A planner is not the normal mechanism for coding-assistant source generation.

2 Project setup and Studio orientation

ZipperGen Studio is the recommended way to experience the development cycle. It is a lightweight, project-aware command environment inside the ordinary terminal. It does not replace the operating-system shell, Git, the editor, or the visible Python workflow. Instead, it remembers the project context and turns the most common operations into short, discoverable commands.

These chapters describe Studio by topic, including optional configuration and migration details. The shorter companion tutorial deliberately omits those branches. Here, the project root contains the canonical specification, generated workflows, tests, and its own Git history; the nested `zippergen` directory contains the framework checkout. Readers who already have both may skip only that first subsection. Section 18 later explains prerequisites and alternatives in more detail.

What is required, and what is optional

The free local path requires macOS or Linux, a terminal, an Internet connection for the initial clone and installation, and enough familiarity to copy commands and edit a file. The built-in mock LLM needs no API key and makes no paid model calls. Codex or another repository-aware coding assistant is needed only to generate or refine source from a prompt; the shipped example is the fallback. OpenAI and Anthropic accounts, real API keys, and supervised service startup are all optional exercises, clearly labelled below.

What Studio removes from the normal development path

For the guided local cycle, you do not need to export `ZIPPERGEN_HOME`, invent a SQLite filename, copy a task ID, keep two terminals synchronized, or remember the inspection flags. Studio creates unique durable stores, presents human tasks inline, and remembers the current workflow, run, view, and named deployment for this project.

2.1 Create the project and framework checkouts

Open an ordinary terminal. Create the example project, initialize its Git history, clone the framework below it, and install the framework's command as an editable user tool:

```
mkdir -p "$HOME/Projects"
cd "$HOME/Projects"
mkdir zippergen-tutorial
cd "$HOME/Projects/zippergen-tutorial"
git init
git clone https://github.com/zippergen-io/zippergen.git zippergen
uv python install 3.12
uv sync --project zippergen --python 3.12
uv tool install --force --editable ./zippergen
zippergen --help
```

The parent is now the tutorial project and coding-assistant root. Its nested `zippergen` directory is a separate framework checkout containing `AGENTS.md`, the workflow skill, framework source, and shipped examples. Studio will create the application project's fixed `specification.md` when you enter `workflow create`; there is no prompt directory or prompt filename to choose. The ignored `.zippergen/pending-refinement.md` is likewise created automatically only when

a change is pending. `uv sync` creates the framework's local environment; `uv tool install --force --editable` makes the short `zippergen` command available while still using this editable checkout. The project, checkout, downloaded Python, and environment survive closing the terminal or rebooting. The final help output should list `studio` among the available top-level commands. You do not run `model` here: `model` is a short command entered later inside Studio.

If your existing tutorial checkout is flat

An older version of this manual cloned ZipperGen directly into `zippergen-tutorial`. In that layout the project root itself contains `AGENTS.md`, `pyproject.toml`, and `src`, and there is no nested `zippergen` directory. Do not add a `zippergen/` prefix to framework paths. The Studio creation command is still simply `workflow create`. For that flat checkout, run these equivalent installation commands from its root:

```
uv sync --python 3.12
uv tool install --force --editable .
```

If `git` is not found, install the standard Git package for your operating system and repeat the first listing. If `uv` is not found but `curl` is available, install it with:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Restart the terminal if the installer requests it, then continue from:

```
cd "$HOME/Projects/zippergen-tutorial"
uv python install 3.12
uv sync --project zippergen --python 3.12
uv tool install --force --editable ./zippergen
zippergen --help
```

If `$HOME/Projects/zippergen-tutorial/zippergen` already contains the framework checkout, do not clone over it. Enter the project and inspect both Git repositories separately:

```
cd "$HOME/Projects/zippergen-tutorial"
git status --short
git -C zippergen status --short
```

An empty result from each command means that repository has no uncommitted changes. Before the first Studio `project init`, the outer command may show `?? zippergen/`; project initialization adds the local framework checkout to the outer `.gitignore`. Do not run `git pull`, delete files, or reset changes you do not understand merely to begin the tutorial.

2.2 The short path and the transparent path

Studio and the detailed CLI operate on the same Python workflows, semantic views, SQLite runtime, and guided deployer. Studio is therefore not a second execution model or a visual builder. The later chapters remain important when you want automation, CI, an explicit store, an external approval adapter, or a precise understanding of every generated artifact.

3 Specification and workflow development

The following topics explain every box in Figure 2 using a reviewed-answer workflow created from a natural-language specification. Studio and the coding assistant handle its Python filename

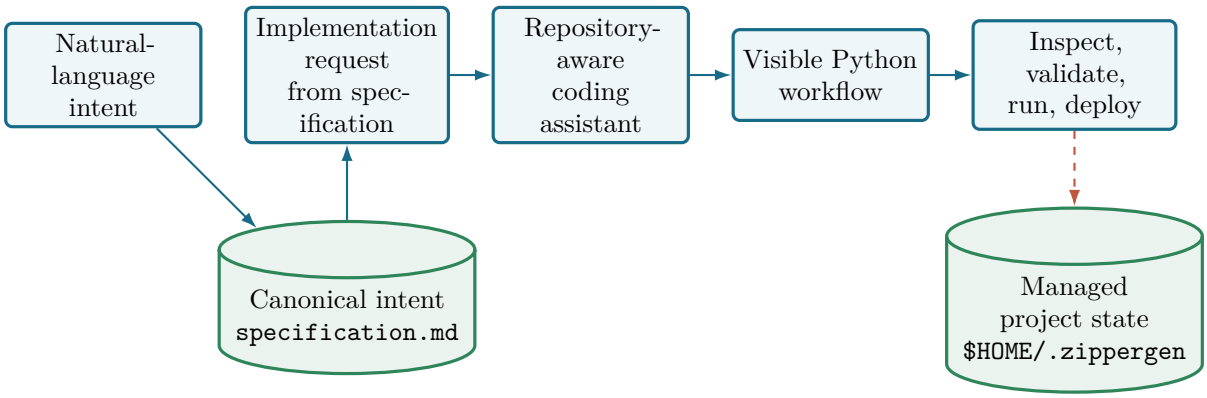

Figure 2: Studio turns natural-language intent into one versioned project specification before preparing a coding-assistant handoff. Visible Python is then inspected by deterministic ZipperGen operations; runs and secrets remain outside the Git checkout.

and symbol. The shipped `zippergen/examples/tutorial_review.py:tutorial_review` is the deterministic fallback if a coding assistant is not available.

3.1 Initialize the project in Studio

Keep the first terminal in the Git root and start Studio:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
```

Expected beginning:

```
ZipperGen Studio
Session context
Project    zippergen-tutorial
Root       /path/to/zippergen-tutorial
Workflow   WARN none selected
Assistant  OK Codex CLI found
Start      Type a command or describe what you want.
           Press Tab to complete. Use 'help' for the short path.
Next
  project init
zippergen [no workflow]>
```

Everything after the last line is a Studio command. Do not prefix Studio commands with `zippergen`. Enter `help` now; it shows the small creation-to-deployment path. Enter `help all` only when the complete exact reference is useful. The assistant line may instead be a warning when neither supported CLI is installed; specification, deterministic inspection, mock execution, and the rest of Studio remain available.

The command vocabulary has four main areas. Everything that changes or explains the application design lives below `workflow`: creation, refinement, coding-assistant implementation, selection, semantic views, validation, review, and history. Everything about model connections, configurations, checks, and assignments lives below `model`. Development execution, decisions, traces, and inspection remain below `run`. Creating and operating an installed application, including its durable state and decisions, lives below `deploy`. Thus a beginner can type `workflow`, `model`, `run`, `deploy`, or `current` to recover the next step instead of remembering several unrelated nouns. The small cross-cutting `settings` command covers global Studio preferences rather than application behavior.

The `studio doctor` command checks the local development front door: manifest, editor, coding assistant, and natural-language interpreter. It does not contact OpenAI, Anthropic, Mistral, or a local model endpoint. Codex and Claude are optional external programs with their own installation and login; they are deliberately not Python dependencies of ZipperGen.

Two deliberately explicit process commands cover reloading and updating Studio. Use `studio restart` to replace the running process and import source that is already installed. The replacement keeps the current directory and reloads the project context saved on disk. It does not open a nested Studio.

When Studio is running from the ZipperGen Git checkout, use:

```
studio update
```

This command finds the actual ZipperGen source checkout. It does not assume that the checkout is named `zippergen` or that it is directly below the project directory. It refuses tracked local changes and detached-HEAD state, runs `git pull --ff-only`, synchronizes the environment with `uv` when dependency metadata changed, and then restarts the same Studio session. The application project remains the current project. Existing deployment bundles remain immutable and are not restarted or replaced.

For a packaged installation, Studio explains that the corresponding package manager must perform the update. Do not confuse either Studio process command with `deploy restart [NAME]`, which restarts an installed deployment.

Every newly prepared deployment records the ZipperGen version and, for a source checkout, the Git revision installed into its managed environment. `deploy show NAME` displays that runtime provenance. It reports a log exception as the current `Cause` only while the current service is failing. An error left by an older deployment generation is labelled `Previous failure`. Preparing a deployment also establishes a new log generation boundary without deleting the earlier log history.

Press Tab at any point while entering a command. Studio completes commands and subcommands and, where relevant, offers the workflows, participants, LLM-active participants, providers, remembered deployment, and project files that are valid in the current context. For example, type `wor` followed by Tab to obtain `workflow`; after selecting the tutorial workflow, `workflow show agent W` followed by Tab completes `Writer`. When several matches exist, the completion menu explains each one. When exactly one match exists—for example, `resu` for `resume`—the bottom line shows that match and its explanation even though no menu is necessary. Use the up and down arrows for private per-project command history. A faint suggestion from that history is accepted with the right arrow. This interactive support is provided by `prompt-toolkit`; scripts and piped input keep the plain non-interactive input path.

Initialize the visible project contract, then inspect the initial dashboard:

```
project init zippergen-tutorial
current
```

Studio creates `zippergen.toml` and safe Git ignores for `.zippergen`, the nested framework checkout, and the optional transparent runtime. The manifest and canonical `specification.md` are project source and may be committed. Timestamped tasks, semantic baselines, runs, and secrets remain in private Studio state outside the project. Studio mirrors only the current generated task at `.zippergen/current-task.md`; Git ignores that file. Repeating `project init` is safe; it reports the existing manifest instead of overwriting it. Use `project rename "NEW NAME"` to change the logical name later. This edits only the versioned manifest: the directory, Git checkout, workflows, deployments, and private workspace identity remain unchanged.

If you later need a reset, do not delete hidden directories or visible project files by hand. Plain `project reset` first asks which scope you mean: *Fresh design cycle*, *Studio state only*, or *Cancel*. The state-only choice archives runs and stores, assistant-task/refinement/command history, model/provider preferences, development secrets, generated tasks, and pending refinement while retaining every visible project file. The fresh choice additionally archives `zippergen.toml`, `specification.md`, and legacy prompt material. Both keep workflow source, tests, Git history, the nested framework checkout, and deployments. Archives are owner-only below `ZIPPERGEN_HOME/resets`. Neither reset scope changes the global preferences shown by `settings`.

This distinction explains the complete restart sequence. Choose the fresh design cycle, confirm it, then enter `project init` and `workflow create`. The manifest is genuinely new and the specification contains only the writing guide described below, not the previous requirements. For automation, the explicit commands are `project reset fresh --yes` and `project reset state --yes`; ambiguous `project reset --yes` is rejected.

How Studio reports outcomes

A green ✓ means an operation or check succeeded. A yellow warning sign means setup is incomplete or needs attention, but is not necessarily broken. A red × means an operation or validation check failed. The symbol always carries the meaning, so the output remains clear without color. Studio adds color only in an interactive terminal; redirected output and sessions with `NO_COLOR` remain color-free. Optional providers that you are not using are warnings rather than failures.

Every nonempty command also starts a clearly separated output block. For example:

```
zippergen [reviewed_answer]> current

+------------------------------------------------------------+
| ZipperGen Studio : current                                 |
+------------------------------------------------------------+
Current
-------
```

The real terminal uses connected curved Unicode corners, a Unicode vertical bar, and a middle dot; the connected ASCII box above is print-friendly. The banner repeats only the command family, never its prompt text, paths, model values, or secrets. It therefore remains short even for a long `workflow create` or `workflow refine` command. Blank input and `exit` produce no banner. Use `settings set output compact` only if a one-line boundary is preferable.

Inside the banner, Studio uses the same hierarchy for every command: a section title and rule, explicit column headings, another rule separating those headings from the actual rows, and a standalone `Next` section when guidance is useful. Success, warning, or failure appears before the data it describes. Thus `project`, `workflow`, `model`, `language`, `run`, and `deploy` results do not require the reader to learn different output layouts. Long fields and table rows wrap to the terminal width, so a path, assistant-check command, or assistant explanation no longer extends far beyond its heading. Wide multi-column data lists may use more terminal width than prose key/value tables. Identifier and timestamp cells remain one line and shorten with an ellipsis instead of being split into uncopyable fragments; the corresponding `show` or `path` command exposes the exact value. The managed `run` and `resume` commands use this same renderer; they do not switch to a second set of bare runtime messages.

Studio accepts exact commands as well as ordinary-language requests, and it discovers a usable terminal editor automatically. Neither feature needs configuration before the first specification. The language, learning, settings, and editor controls are collected later in Section 11; continue directly to specification authoring.

3.2 Write the canonical specification

Long requirements do not belong in a single-line terminal question, and the user should not have to invent or remember a prompt filename. Enter `workflow create`. Studio creates the fixed project file `specification.md`, opens it in the automatically discovered editor, and waits. On the first creation it contains a short comment explaining what a specification should cover and which development mechanics to omit. Write below that comment or replace it with the complete text below. Studio removes the guide automatically after real requirements are saved; leaving the guide untouched does not create an assistant task. Copying into an editor is reliable even when a terminal's interactive question cannot accept a multiline paste.

```
workflow create
```

```
# Reviewed Answer

Produce a helpful answer to a request, subject to automated review and explicit
human approval.

## Participants and behavior

- Requester initially owns `request` and `max_retries`.
- Writer uses an LLM to draft and revise an answer.
- Reviewer uses a separate LLM to assess every draft.
- HumanApprover owns the decision to approve or reject a draft after the
  automated assessment.
- A rejection sends the draft back to Writer while retries remain.
- Never return an unapproved draft. Return an explicit failure after retry
  exhaustion.
- Keep human delivery independent of workflow source. HumanApprover's
  `@human` actions use the terminal by default and may later be assigned a
  reusable Telegram configuration.

## Inputs and operation

- The application accepts a `request` input, defaulting to
  `Explain why the sky is blue in two sentences.`
- It accepts an integer `max_retries` input, defaulting to `2`.
- Use the deterministic mock model by default so the application can be tried
  without a paid model account.
- A deployment may instead use OpenAI or Anthropic. Request the corresponding
  API key only when that provider is selected.
```

Save the file and exit the editor. Studio reads the whole UTF-8 document, preserves its paragraphs and lists, and wraps it in repository-aware instructions. It prints a compact table instead of repeating the long text:

```
+-----------------------------------------------------------+
| ZipperGen Studio : workflow create                        |
+-----------------------------------------------------------+
Creation
  Specification  [success] specification.md
  Implementation [success] prepared
  Next                     workflow implement codex / workflow implement claude
  Inspect                  workflow status / workflow history
```

There is no prompt ID, numbered prompt file, or ordering operation. The canonical document is always `specification.md`; it is ordinary project source and should be reviewed and committed with

the Python workflow. The generated, non-secret handoff is always `.zippergen/current-task.md`; Studio also keeps timestamped private task history outside Git.

Projects created by an older Studio may still contain an ordered prompt ledger. On first specification access, Studio combines its active entries into `specification.md` and preserves the old files as migration history. The former `workflow prompts` command is deliberately unavailable: all new work uses one canonical specification and at most one pending refinement.

The implementation request asks the assistant to identify participants, ownership, messages, action kinds, decisions, loops, deployment needs, safety assumptions, and success examples before editing. It also requires tests, validation, semantic views, and all exact local projections. No workflow file has been created yet; the implementation subsection starts the coding assistant on this request.

Notice that the specification does not choose a Python filename or symbol, request tests and validation, or repeat the prohibition on deployment. Those are development mechanics supplied automatically by Studio's generated task. The specification contains only durable application intent. After generation, Studio discovers the resulting workflow so the user can list and select it with `workflow list` and `workflow select` without knowing its source path in advance.

The everyday specification commands are intentionally uniform:

```
workflow show spec    # read the canonical requirements
workflow edit spec    # reopen the same specification.md
workflow path         # print its absolute path only when another tool needs it
workflow status       # inspect the implementation lifecycle
workflow status --details # add task IDs, refresh links, and record paths
workflow history      # list specification and implementation history
```

The ordinary status and `current` dashboards intentionally hide generated task filenames, request IDs, and semantic-baseline paths. Those are auditable plumbing rather than concepts required for daily work. `workflow status --details` and `workflow history` expose them when diagnosis or traceability requires them.

An empty specification is rejected. Never put an API key or other secret in it. For a genuinely short experiment, `workflow create DESCRIPTION` writes the same canonical file without opening an editor. Advanced users may import an existing UTF-8 document once with `workflow create --file PATH`; Studio copies its content into `specification.md` and does not retain that source filename as project configuration.

Studio fingerprints the canonical specification when it prepares a task. While the implementation is still prepared, Studio compares that fingerprint before `workflow status`, `current`, or `workflow implement`. If an input document changed, Studio creates one refreshed private request and records which older request it replaces. After an assistant has run, its expected edits are implementation output, not new task input: Studio preserves the executed request instead of silently creating another "ready" task.

Projects made with an older Studio may still contain `prompts/index.toml` and numbered prompt files. On the first `workflow show spec`, `workflow edit spec`, or `workflow refine`, Studio safely combines their active contents into `specification.md` and keeps every old file as history. New work then uses only the canonical specification model.

Why Studio creates a brief instead of hidden code

Natural language is flexible; executable coordination must be precise. The brief supplies repository rules and completion checks. The resulting ordinary Python file is inspected and versioned. ZipperGen never treats opaque model output as an automatically deployed workflow.

3.3 Implement the specification with a coding assistant

At the same Studio prompt, enter:

```
workflow implement codex
# Or, when Claude Code is installed:
workflow implement claude
```

Choose one of the two commands; do not run both for the same implementation request. After `settings set assistant codex` or `settings set assistant claude`, bare `workflow implement` uses that global preference. Studio runs the selected local coding assistant once in the project root and tells it to read `.zippergen/current-task.md`. Just before launch, Studio applies the freshness check described above, so the file contains the current canonical specification. There is no task path or prompt file to copy. The assistant autonomously inspects and edits the application project within its configured permission boundary, runs checks, writes a small structured assistant-check handoff, exits, and returns control to the same ZipperGen Studio session. The handoff distinguishes the assistant process returning from the requested checks actually passing. Use `workflow implement codex --interactive` only when a conversation or interactive approval is genuinely needed.

The generated task also explains the tutorial's two-project layout. The application lives at the tutorial root, while `zippergen/pyproject.toml` defines the environment for the nested framework checkout. It therefore tells the assistant to invoke the framework from the application root with `uv run --offline --project zippergen zippergen ...` and to run application tests by the explicit `tests` path. A bare `uv run pytest` at the tutorial root is deliberately forbidden: it can collect `zippergen/tests` while using an environment that did not install the framework dependencies. The framework's own suite is separate and is needed only when framework source was actually changed. Pytest is a declared development dependency installed by the initial `uv sync`; the assistant must not add `--with pytest`, install packages, or request PyPI access. If the local environment has not been synchronized after an update, run `uv sync --project zippergen` once in the ordinary terminal and then rerun the assistant deliberately. Studio checks this prerequisite offline before starting Codex or Claude, so a missing pytest installation does not consume an assistant run.

For one-shot Codex runs, Studio uses Codex's structured JSON output mode and shows the final assistant report rather than its raw execution trace. Known internal model-cache diagnostics are suppressed; unexpected diagnostics are condensed into warnings. Studio first prints that Codex is working and then prints periodic elapsed-time heartbeats, so the synchronous wait never looks like a frozen prompt. Control-C preserves the implementation request and any project changes; `workflow status` shows the interrupted state. Interactive Codex sessions retain their ordinary terminal output.

There is no assistant queue or scheduled background job. The command runs synchronously: before launch, `workflow status` says the implementation is prepared and `Execution: not started; nothing is scheduled`. A normal return changes the same request to `implemented` and prints a bounded summary of changed files, assistant checks, and the assistant report. Separately, `Assistant checks` are `passed`, `failed`, or `incomplete`, with a concise note and the number of checks the assistant reported. The `workflow status` output lists each reported command and its outcome underneath the summary. A missing, malformed, or internally inconsistent handoff is never interpreted as success. A failed or interrupted assistant process remains visible and retryable. If Studio or the computer stops while the assistant is running, the orphaned state is recovered as `assistant interrupted` on the next inspection. Studio refuses to rerun an `implemented` request accidentally; use `workflow implement codex --rerun` or `workflow implement claude --rerun` only when a second pass is intentional. Studio refreshes an older unexecuted task automatically before launch so that it also receives this assistant-check contract.

```
workflow status
Workflow implementation
  Status     [success] implemented
  Assistant            Codex -- returned ...
  Execution            assistant session ended; nothing is scheduled
  Assistant checks [success] passed -- 5 checks
  Check summary              Validation, projections, diff, and focused tests
    passed.
  Context    [success] implementation matches this task
  Next                  workflow diff / workflow validate / run / deploy
```

The checks below that summary are records rather than unbounded table values:

```
Assistant checks
----------------
5 checks / 5 passed / 0 failed / 0 not run

[success] 1. passed
  Command  uv run --offline --project zippergen zippergen validate
           workflows/reviewed_answer.py:reviewed_answer
  Result   Validated the global workflow and every local projection.
```

Studio preserves the complete command and result but wraps them with hanging indentation. Assistant-check records are capped at 108 columns and use a narrower interactive terminal width when necessary. A single long check therefore cannot create a screen-wide separator. When assistant checks failed, failed and unexecuted records are shown before successful checks.

The assistant-check result is an explicit assistant report, not an independent proof. If it is `failed` or `incomplete`, Studio shows that state prominently and proposes inspection, validation, and an intentional assistant rerun instead of implying that the change is ready. Immediately below the summary table it prints only failed or unexecuted checks; the complete reported check list remains available through `workflow status`. Use `workflow diff` to inspect changes again and `workflow validate` for ZipperGen's technical check of the current workflow.

The internal handoff retains a field named `verification` for its versioned JSON contract. Studio presents it as assistant checks so it is not confused with `workflow validate`. The summary works in non-interactive sessions. Detailed inspection remains available through `workflow diff`, `workflow show`, and `workflow status --details`.

If Studio says that `codex` is missing, the prepared task remains safe. Open a second operating-system terminal. If npm is already installed, install and authenticate Codex once:

```
npm install --global @openai/codex
codex login
cd "$HOME/Projects/zippergen-tutorial"
codex
```

Then return to Studio and enter `workflow implement codex` again. Section 18.3 explains setup for both supported assistants and the no-assistant fallback in more detail. The task points the assistant to `zippergen/AGENTS.md` and the nested framework's workflow skill when the skill is not installed at the parent project root. The long workflow specification remains in `specification.md` rather than being pasted into the assistant's terminal.

What `workflow implement` is—and is not

`workflow implement codex` and `workflow implement claude` are small launchers for the locally installed Codex CLI and Claude Code respectively. They do not send the task through ZipperGen's Writer or Reviewer model providers and need no ZipperGen API key. Each tool uses its own authentication, model settings, approval policy, and tools. MCP is not required. Independently configured MCP servers remain available to the chosen assistant, but Studio neither creates nor manages them. Integrations can also consume Studio's generated implementation request directly.

Wait for the assistant to report generated source, focused tests, validation, communication-only and full views, and projections for Requester, Writer, and Reviewer. Then inspect the checkout in the ordinary terminal:

```
git status --short
git diff
```

Do not commit merely because code was generated. First complete the checks below. If generation is unavailable or fails, continue with workflow discovery and select the shipped `tutorial_review` workflow instead. The selector displays its internal source reference; the user does not have to type it.

3.4 Discover and select generated workflow source

Return to the Studio terminal and enter:

```
workflow list
workflow select
workflow files
workflow show source
```

Studio displays the discovered workflows. Select the newly generated reviewed-answer workflow by its number; there is no path or Python symbol to type. The selected entry may display an internal reference of the form `PATH.py:NAME`: the suffix names the Python workflow object, not a deployment. Inspect the authored Python before asking validation to judge it. The commands have deliberately different jobs:

`workflow list` discovers top-level `@workflow` entry points from project source and lists their `PATH.py:NAME` references. This is source discovery, not validation.

`workflow import PATH.py[:WORKFLOW]` copies an external workflow into the current project before selection. It preserves paths relative to the external project root when that root has `pyproject.toml`, `zippergen.toml`, or `.git`. It also copies statically imported local Python modules and literal files named by deployment or assistant declarations. It never overwrites a different project file.

`workflow select` loads the chosen entry point and remembers it as the current workflow for later Studio commands. It neither validates, accepts, runs, nor deploys it.

`workflow files` lists the entry module, statically imported project-local Python modules, and declared resource files known to the selected workflow.

`workflow show source` displays the authored entry module. Use `workflow show source NUMBER` or a path printed by `workflow files` to inspect another listed file.

A workflow entry point need not occupy one file. It may import `@fragment` definitions, actions, connectors, or helpers from other modules. Conversely, one module may expose several top-level `@workflow` entry points. Studio lists entry points as choices; supporting files are inspection material, not additional workflows.

If no workflow is selected, `workflow show`, `workflow files`, `workflow edit code`, and `workflow validate` perform this selection step automatically. One discovered entry point is selected with an explicit notice; several open the numbered selector. In both cases Studio says that validation has not yet run.

4 Workflow inspection and validation

4.1 Inspect increasing degrees of detail

Enter `workflow show` and choose the semantic views:

```
+-----------------------------------------------------------+
| ZipperGen Studio : workflow show                          |
+-----------------------------------------------------------+
1. Authored source
2. Overview
3. Protocol
4. Communications only
5. Actions and prompts
6. Complete workflow
7. One participant
8. Selected participants
```

Then use the direct forms:

```
workflow show communications
workflow show actions
workflow show full
workflow show agent Writer
workflow show agent Reviewer
workflow show agents Writer Reviewer
connector
workflow validate
current
```

All results are code. `workflow show communications` hides local actions and retains messages and control. `workflow show actions` reveals action calls and prompts. `workflow show full` adds implementation and deployment requirements. `workflow show agent Reviewer` is the exact projected local program. `workflow show agents Writer Reviewer` is a focused global view with hidden peers marked as boundaries; it is not a projection. `connector` discovers the human-active participants and actions from the workflow. Before an external route is assigned, it shows the local terminal as their effective delivery path.

Only the final `workflow validate` command makes a validity claim. Discovery answers which entry points exist, and selection answers which one is being discussed. Validation additionally checks the global protocol, input ownership, every local projection, action metadata, referenced resources, and deployment declarations. Every successful check begins with a green check mark; a red cross identifies a failure and is followed by its explanation. `current` then shows the active workflow and execution context, including the fresh validation summary, model routing, latest run, and selected deployment. Use `project` when you want the complete project inventory.

Validation and deployment answer different questions

`workflow validate` checks the current code: is the global workflow structurally valid, and can every participant be projected consistently? It can be run repeatedly and does not change project state.
When a local implementation record exists, `deploy` checks a separate coherence property. Studio compares that record's specification fingerprint with the current fingerprint. If they differ, run `workflow implement` again before deploying. A fresh clone has no portable record yet, so Studio warns and proceeds after technical validation. Git remains the version history for source and specification changes.

Inspect declared boundaries in proportion to their risk. A workflow with human actions, external effects, assistant actions, or deployment metadata deserves closer source and projection inspection. Model calls can transmit data, and prompt changes can materially change behavior even when no `@effect` is present.

Before running, confirm from these views who owns both inputs, who owns the retry decision, whether every revision returns to human review, whether an unapproved draft can reach Requester, whether both model prompts are visible, and whether HumanApprover's human action is discoverable. Do not continue until the code answers all six clearly.

If the connector view reports no human actions, inspect the full workflow. The generated code is incomplete when the requested `@human` action is absent. After implementation, use `workflow implement codex --rerun` or the corresponding Claude command. Record a requested correction with `workflow refine`, then implement the updated specification again.

5 Model configuration and routing

5.1 Configure models without shell exports

Enter:

```
model default mock
model
model config check mock
```

Studio lists only lifelines containing `@llm` action sites, together with their action names. Requester is absent because message passing and pure validation do not require a model. Expected shape:

```
Provider connections
--------------------
  Provider     State              Details
  --------     -----              -------
  mock         built in           ... available; built in
  local        not configured     ...

Secrets
-------
API-key values are never displayed or written to the project.

Model configurations
--------------------
  Name                   Provider    Model                       Status
  ----                   --------    -----                       ------
  mock                   mock        built in                    ... available
```

```
Model assignments
-----------------
  Participant  Configuration   Model  LLM actions                Last check
  -----------  -------------   -----  -------------------------  ----------
  Writer       mock (default)  mock   draft_reply, revise_reply  ... built in
  Reviewer     mock (default)  mock   assess_draft               ... built in

Execution
---------
  ... Configurations can be shared; calls remain independent and may run in
    parallel.
```

Generated action names may differ, but Writer and Reviewer should appear. The non-secret assignments are stored per workflow and survive a reboot. The built-in `mock` configuration needs no provider connection and is already checked. Plain `model assignments` uses saved check state and performs no network request. Enter `model assignments check` to check only the effective configurations shown in this table; shared configurations are checked once.

6 Durable execution, recovery, and stores

6.1 Make a durable mock run in one terminal

Enter `run`. Before asking for inputs, Studio checks every model configuration that is actually used by an LLM-active participant and performs technical workflow validation. A shared configuration is checked once. If a provider is unreachable or the configured model is absent, Studio stops before it creates the run and points to the relevant `model config check` command. The run summary shows one effective model route per LLM-active participant rather than only showing the default fallback.

```
zippergen [reviewed_answer]> run
Run model checks
...
Workflow reviewed_answer: valid
Request to answer (str) ["Explain why the sky is blue in two sentences."]:
Maximum revisions after the initial draft is rejected (int) [2]: 3
Run reviewed_answer-<timestamp>

Draft:
[draft_reply:draft]

Automated reviewer notes:
[assess_draft:concerns]

Approve this draft? [y/n]: n
```

Press Enter for the displayed request, enter `3` for the retry budget, `n` for the first review, and `y` for the second. The mock result is a recognizable placeholder. The important behavior is a separate Writer draft, Reviewer assessment, visible human authority, and another revision and assessment after rejection.

Enter `current` and `runs`. Studio shows a unique managed SQLite store even though no filename was typed. Every new `run` receives a new store and never silently reuses an old result.

6.2 Inspect current projected positions

After an interrupted run, enter at the Studio prompt:

```
run inspect
run inspect Reviewer
```

For a live foreground run, keep its terminal untouched and open another terminal at the project root:

```
zippergen
run inspect Reviewer --watch
```

The second Studio process performs read-only inspection. It refreshes the positions once per second in one fixed terminal screen, without appending repeated output. Its differential renderer updates only changed terminal cells, so pointer movement does not clear and repaint the complete screen. Press Ctrl-C to restore the Studio screen and return to its prompt. The development run is not interrupted. For a single non-interactive snapshot, use:

```
zippergen studio --command "run inspect Reviewer"
```

The first process continues to own terminal input and any inline human decision. The first command reads the current development run's durable observation rows and shows one participant per line. States distinguish running, receive-blocked, human/model/assistant/effect activity, completion, cancellation, and failure. Studio focuses the participant most likely to need attention. Naming a participant focuses it explicitly and prints its exact projected local program with one or more pointer markers. Multiple pointers are possible inside a local parallel region.

This view has size proportional to the projected program, not the accumulated trace: a hundred loop iterations move the same pointer instead of adding a hundred message rows. Observation locators are deliberately separate from the loop-boundary snapshots used for recovery. Studio also withholds local environment values, action inputs, and secrets; use task presentation or explicit trace inspection when those details are genuinely required.

6.3 Work through the current development run

Enter:

```
run inspect
run tasks
run approve
run trace
```

`run inspect` shows the current participant positions and one focused local projection. Bare `run approve` selects the sole pending task automatically or opens a numbered selector when there are several. `run tasks` renders each task's instruction and decision context, and `run approve` repeats the complete context immediately before collecting a response. For the reviewed-answer workflow, that context contains the proposed draft and automated reviewer notes, so the human never approves an opaque identifier. `run trace` shows recent durable events.

Studio creates a unique SQLite file for each development run. The file is an implementation detail, not a separately selected or renamed Studio object. The corresponding deployment commands operate on the one durable state owned by a named deployment.

6.4 Verify crash and resume behavior

Start another run and press Control-C at the human prompt. Studio records the managed run as interrupted and prints a recovery hint instead of a traceback. The expected transition is:

```
Approve this draft for the requester? [y/n]: ^C
Command interrupted. Use 'current' to inspect context; use 'resume' for an
    incomplete managed run.
zippergen [reviewed_answer]>
```

Only Studio's main thread reads the terminal. After the warning, no abandoned workflow thread should consume a Studio command or print another approval prompt. You may enter `current` and `resume` immediately; `current` reports the run as interrupted, and `resume` presents the same still-pending approval once. To test a full terminal or computer restart, leave Studio, reopen a terminal later, and enter:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
current
resume
```

`resume` reuses recorded inputs, model routing, and SQLite state; the pending task is presented again. It also checks the workflow's semantic fingerprint. If source meaning changed, Studio preserves the store and refuses an unsafe resume.

No export must be repeated. The workflow, model routing, run record, and development secrets live below `$HOME/.zippergen/workspaces` by default. A shell export disappears with its shell; Studio's workspace record does not.

The initial workflow has now been generated. Inspect its differences and run the technical check:

```
workflow diff
workflow validate
```

`workflow status` should say `implemented`. The implementation summary already showed changed files, assistant checks, and the assistant's report. `workflow diff` is the detailed inspection path, while `workflow validate` independently checks structure and projection.

7 Real-model readiness and participant routing

Only do this after the mock path succeeds and only if you have provider accounts, valid model access, and permission to incur API charges. Otherwise skip directly to refinement; the workflow remains fully usable with mock models.

If you choose this optional step, prepare each provider account before changing the Studio profile:

1. Sign in to the provider's API console, not merely its consumer chat application. OpenAI keys are managed at `https://platform.openai.com/api-keys`; Anthropic API access and keys are managed through `https://console.anthropic.com`.
2. Select or create a development project/workspace and confirm that the account has API billing or credits. API usage may be billed separately from a consumer chat subscription; do not assume the latter funds it.
3. Set a small development budget, usage alert, or credit balance where the provider supports it. Confirm that the intended model is enabled for that project/workspace.

4. Create a separate development key. A provider may show the complete secret only once. Keep it private and available only long enough to paste it into Studio's hidden prompt below. Do not type it into a shell command to test it.

See OpenAI's current API-key location and API-key safety guidance. If a key is lost, pasted into the wrong place, or appears in Git output, revoke it in the provider console and create a replacement before continuing.

```
model
model default mock
model provider configure openai
model config create drafting
model config check drafting
model assign Writer drafting
model provider configure anthropic
model config create careful-reviewer
model config check careful-reviewer
model assign Reviewer careful-reviewer
model
```

The model workflow has three deliberately separate layers: *provider, configuration, assignment.* A provider connection contains an API key or local endpoint. A named configuration contains a provider and exact model identifier. For a local Ollama model, it can also contain an idle-release time. An assignment maps an LLM-active participant to one of those names. Enter `model setup` for a guided pass through all three layers.

`model provider configure PROVIDER` asks for a missing API key without echo, or for the endpoint of a local provider. It saves and checks the connection. `model provider check PROVIDER` repeats that check later. `model config create NAME` then asks for one configured provider and one model identifier. It does not collect credentials: if the chosen provider is not configured, Studio stops and prints the exact provider command to run first. Plain `model` makes no remote request, never prints a secret, and displays provider connections, named configurations, and workflow assignments as separate tables.

`model config check` checks every saved configuration; `model config check NAME` checks one. It records `available`, `unverified`, or `unavailable`, but never changes which participant uses which model. A green result confirms that the provider accepted the configured key and exact model identifier at check time. A yellow result means connectivity prevented a conclusive answer. `model assignments` displays the effective route of every LLM action, its source, and its cached last-check state. It never contacts a provider. `model assignments check` performs a fresh, targeted check of only those effective configurations, deduplicating configurations shared by several participants. `run` repeats that targeted check as a mandatory preflight. For a local OpenAI-compatible or Ollama endpoint, `model provider configure local` asks for the endpoint (normally `http://127.0.0.1:11434/v1`). A later `model config create NAME` asks for the local model. Local identifiers are checked against the live `/models` response. Studio also asks when the local model should be released after its last call. The default is `300` seconds. Enter `never` to keep it loaded, or `0` to release it after every call. The workflow and its durable state remain active. Ollama loads the model again automatically when the next LLM action is reached. The policy follows the named configuration into development runs and deployments. Hosted providers need no idle-release setting because ZipperGen does not keep their models in local memory.

`model assign PARTICIPANT NAME`, `model assign PARTICIPANT.ACTION NAME`, and `model default NAME` only assign saved configurations; they do not contact the provider. Studio warns when a configuration has not yet been checked but allows the assignment so offline preparation remains possible. A configuration whose last conclusive check found it unavailable must be fixed or checked

successfully before assignment. An action assignment takes precedence over its participant assignment, which takes precedence over the default. Enter `model inherit PARTICIPANT.ACTION` to remove one action override, or `model inherit PARTICIPANT` to restore the project default for that participant. Use Tab completion rather than retyping generated names. Editing one named configuration with `model config edit NAME` updates every assignment that refers to it. `model config rename OLD NEW` changes only its reusable name: Studio preserves the provider, model, and recorded check result and updates every default and participant assignment in the project atomically. It does not reconnect to the provider or change runtime model selection. `model config remove NAME` refuses to remove a configuration that is still in use. `model provider remove NAME` removes a provider connection and its privately stored key.

One named configuration may be assigned to any number of participants. They share the provider/model settings, but not one stateful conversation, client lock, or execution slot. Their LLM actions remain independent and can run in parallel; the provider's own rate and concurrency limits still apply. The model identifiers above are examples; use models enabled for your provider accounts when necessary.

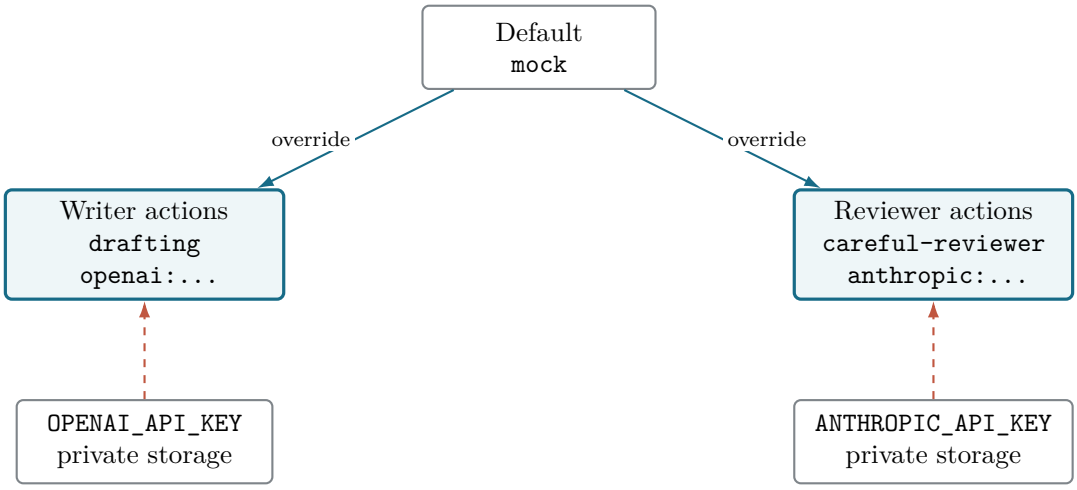

Figure 3: One workflow assigns reusable named configurations to LLM-active lifelines independently. Provider/model specifications are non-secret; provider keys remain separate secrets.

The model dashboard and connection commands display lines such as:

```
OPENAI_API_KEY:
ANTHROPIC_API_KEY:
anthropic    checked    ... last check succeeded; 8 models; checked <time>
```

The provider command immediately attempts a model-list check. A successful line begins with a green check mark; a saved key whose check failed is clearly distinguished from an unconfigured optional provider. Paste each key only at the hidden Studio prompt—never into source, a shell command, an assistant prompt, or a Git-tracked `.env`. Development secrets are mode-0600 and absent from workspace, run, and request JSON. Existing environment values are used but not copied. Later development runs temporarily inject the selected connected providers and restore the process environment afterward. Use `model provider remove NAME` to remove a saved connection and private key.

Use `model inherit Writer` to restore inheritance for one role, and `model default mock` to select the declared tutorial default. Named configurations are a Studio convenience; the scriptable equivalent still uses the resolved provider/model specification with repeatable `--llm-for`:

```
zippergen dev workflows/reviewed_answer.py:reviewed_answer \
  --llm mock \
```

```
    --llm-for Writer=openai:gpt-4o-mini \
    --llm-for Reviewer=anthropic:claude-sonnet-4-6
```

Two different meanings of "change the model"

For a routing-only change, create or edit a named configuration, check it, then use `model default NAME` or `model assign PARTICIPANT NAME`. Studio remembers that choice for runs and deployment; source code does not change, so no refinement or coding assistant is needed. If the model choice must become versioned design intent, change declared workflow/deployment defaults, or accompany prompt, action, or test changes, record it as a refinement and run an assistant. For example:

```
workflow refine Use openai:gpt-4o-mini for Writer and preserve all protocol
    behavior.
workflow implement claude
```

The second form appends to the one pending refinement and asks the assistant to update the canonical specification and visible project files together. It is intentionally heavier than routing.

8 Refinement and reimplementation

8.1 Refine through one pending change

Use `workflow refine` to describe one change to the selected workflow:

```
workflow refine "Require a second review after a rejected draft"
workflow show pending
workflow implement
```

Studio keeps at most one pending refinement. Repeating `workflow refine` updates that request instead of creating a numbered ledger. The coding assistant receives the canonical specification, the pending change, and the semantic baseline captured before the refinement. It must integrate the change into `specification.md`, update source and tests, and preserve behavior outside the request.

For the common chained form, use:

```
workflow refine "Add a retry limit" --implement
```

After the assistant returns, Studio prints an implementation summary with the changed files, assistant-check result, and assistant report. The detailed inspection commands remain available:

```
workflow status
workflow diff
workflow validate
```

`workflow diff` compares the available specification and semantic baseline. `workflow validate` is the independent structural and projection check. If checks failed or were incomplete, inspect their details with `workflow status --details` and rerun the assistant deliberately with `workflow implement codex --rerun` or the corresponding Claude command.

`workflow discard` archives and clears the pending refinement request. It does not revert changes already present in the working tree. Inspect `git diff` and restore unwanted source or specification edits with ordinary Git operations.

When an implementation request is available locally, its recorded specification fingerprint must equal the current specification fingerprint. Editing the specification after implementation therefore blocks `deploy` until `workflow implement` runs again. When no record is available, such as on a fresh clone, Studio warns and proceeds after technical validation. Git is the version history for returning to an earlier source/specification pair.

9 Deployment and operation

First prepare the named deployment without starting a background service. This command snapshots the current working tree into an immutable bundle, creates a separate deployment bundle and deployment-specific Python environment, writes service definitions, and runs readiness checks. It changes files below ZipperGen's private home but deliberately defers service-manager state. On the tutorial machine, enter in Studio:

```
deploy reviewed-answer-tutorial --no-start
```

Studio compares any local implementation fingerprint with the current specification fingerprint. If the specification changed after implementation, deployment stops with `workflow implement` as the remedy. If no local record exists, Studio warns instead. In both cases Studio validates the selected workflow again before preparing the bundle.

A previously running deployment is not reading the working tree: it continues to execute its own timestamped bundle until an explicit deployment or restart changes its operational state. This coherence check belongs to Studio. The lower-level operating-system command `zippergen deploy PATH:WORKFLOW` remains an explicit expert interface and does not consult Studio's implementation request.

Studio carries the current default, participant, and action routes into the deployment profile. If the current model profile selects a real provider and its key was configured earlier, Studio displays a *Deployment credentials* summary containing the key's name but never its value:

```
Deployment credentials
  Available   [success] OPENAI_API_KEY in private Studio storage
  Deployment             reviewed-answer-tutorial
  Storage                separate private deployment secret file

Reuse the configured credential for deployment reviewed-answer-tutorial? [Y/n]:
```

Press Enter to reuse it. Studio copies the hidden value directly into the deployment's separate mode-0600 file, so the guided deployer does not request the key again. This applies to every selected OpenAI, Anthropic, or Mistral route even if the workflow's deployment declaration did not explicitly name that provider's key. Readiness refuses to start when a selected API model has no deployment-scoped credential. Enter `n` only when this deployment should use a different credential; the deployer then asks for that value without echo. A later `deploy reviewed-answer-tutorial` retains the deployment's existing key and prints that it is keeping the saved value, without presenting another secret prompt. A selected local configuration needs no key; Studio copies its configured OpenAI-compatible endpoint into the deployment profile so that the deployed service uses the same local or tunneled server as development. With the recommended `model default mock` and `model inherit` commands above, all routes are mock and no provider key is needed.

Read every prompt. The request and retry budget are runtime inputs, not another description of the workflow. A successful ending prints `Deployment: reviewed-answer-tutorial` and leaves the service stopped. Studio remembers the deployment name. Check the prepared artifacts before the first real service-state change:

```
deploy doctor
deploy show
```

`deploy show` separates four objects that must not be confused: the immutable bundle, the supervised operating-system process, the workflow run, and its SQLite store. Studio condenses the deployer's checks into one readiness summary; `deploy doctor` is the explicit expanded view. Doctor should contain no failures. Deployment preparation initializes the stable store, even with `--no-start`. Before first execution, that store is ready and empty, while a missing log and an uninstalled service are expected warnings. If the checks are acceptable and service startup is authorized, enter these one at a time:

```
deploy start
deploy show
deploy inspect
deploy inspect Reviewer
deploy inspect Reviewer --watch
deploy tasks
deploy approve
deploy logs
deploy restart
deploy show
deploy stop
deploy storage
deploy storage compact
```

`deploy start` is the first command here that changes service-manager state. It reruns readiness checks immediately before doing so; `deploy start` in Studio also enables persistent automatic startup. The `Boot` row of `deploy show` states whether the deployment will start at server boot, after the user logs in, or only after a manual start. On Linux, automatic startup at server boot requires both an enabled systemd user service and account lingering. Studio reports these separately because enabling a deployment does not grant permission to change the server's account policy. `deploy restart` applies the same gate. A failed credential, workflow import, bundle, package, or workflow-specific doctor check blocks the operation before launchd/systemd is touched. Warnings remain informative and do not block startup. `deploy stop` is never subject to this gate. The process may need a few seconds to reach its first action; repeat `deploy show` once if necessary. The run/store rows should report `waiting` once durable human approval is pending. Immediately after `deploy start`, the run may briefly report `starting, no durable events recorded yet`. The separate store row remains ready. Each deployment name owns one stable logical store, so normal deployment operation requires no store selection or copied path. `deploy inspect` reads per-participant positions from that store and projects the workflow from the immutable deployment bundle, so later working-tree edits cannot change what is displayed. With no name it uses the remembered deployment; a participant can be supplied as the final argument. Add `--watch` to refresh the positions and focused projection once per second. Ctrl-C closes only the live view and does not stop the deployment. `deploy tasks` shows the full stored instruction, draft, reviewer notes, and decision labels; `deploy approve` repeats them immediately before accepting yes or no. `deploy trace` shows recent durable events without requiring a SQLite path. `deploy storage` shows the store, WAL, logs, event counts, recovery snapshot coverage, and the amount of history that can be removed safely. It first runs SQLite's structural quick check. The explicit `deploy doctor` command runs the same check and reports a failure if the store is malformed. Normal start and restart readiness checks do not rescan a large healthy store. Diagnostic traces are pruned online in batches. The target is 10,000 traces and the store never keeps more than 10,999. This requires no service stop. Completed human tasks, tokens, and notifications remain as audit records. `deploy logs` shows the service output and returns; `deploy logs reset` archives the

currently visible history and begins a fresh logical log generation without stopping the service or changing its workflow run and durable store. Studio asks for confirmation and reports the private archive path. New output continues in the active log normally. `deploy storage compact` is the durable cleanup command. Stop the deployment first. It deletes only completed communications and journal entries covered by durable recovery floors, rotates the active log, and keeps three log archives. Participants without a recovery snapshot are named and block core event cleanup for the events they still need. Trace retention runs independently while the service is online. Studio then compacts the database file and truncates the WAL while preserving the stable event identifiers used for recovery. Deployments created before this feature must be redeployed once before safe compaction is enabled. Studio reports this instead of touching an older runtime bundle.

Keep durable stores private and on local storage

ZipperGen creates a store with owner-only mode `0600`, uses WAL, and explicitly requests `synchronous=FULL` on every durable protocol writer connection. The store must remain on a local filesystem with reliable SQLite file locking and `fsync`. Do not put it on NFS, a synchronized folder, or a remote container volume whose SQLite guarantees are unknown. Do not edit rows with a SQL client.

The quick check finds structural database damage. It cannot detect a row that was deliberately deleted from an otherwise valid database. Compaction is the only supported row-deletion path. It uses participant recovery floors and may legitimately remove completed rows. Therefore, an always-increasing row count is not a valid integrity test.

The alpha does not yet have a managed restore command. To make a consistent online backup, take the store path from `deploy storage`, choose an owner-private destination, and use SQLite's backup command:

```
mkdir -p "$HOME/.zippergen/backups"
chmod 700 "$HOME/.zippergen/backups"
(
  umask 077
  sqlite3 "/PATH/TO/DEPLOYMENT.sqlite" \
    ".backup '$HOME/.zippergen/backups/DEPLOYMENT.sqlite'"
)
```

This backup may be taken while the deployment runs. Do not restore it by copying over a live store. A backup contains workflow data and human approval tokens, so keep it mode `0600` and outside Git. A future managed restore flow will stop the service, verify the backup, archive the current store, and state the recovery window before replacement.

Restoring an older store can repeat external work that succeeded after the backup but whose journal entry is absent from the backup. This includes model calls, email sends, sheet appends, and other `@effect` or connector operations. Use stable idempotency keys or idempotent remote operations whenever duplicates matter. `deploy restart` deliberately proves that the same pending task survives a process restart. Then `deploy stop` prevents this finite tutorial workflow from being supervised indefinitely. Later chapters show task tokens and notification adapters for delivering the same approval context through Telegram or another out-of-process channel. Stopping does not delete the profile, bundle, log, secrets, or store. The generated launchd/systemd service retries failures but does not relaunch a finite workflow after successful completion.

If Studio reports that no supported user service manager is available, stop at the error and use the later `deploy --no-start` plus `run-deployment` foreground path. Do not repeatedly retry a failing service start without reading `deploy doctor` and `deploy logs`.

Deployment is a real operational action

First complete mock execution and inspect `workflow show full`. A real model call may cost money and an `@effect` may change an external system. Use fake services until effects and retry safety have been reviewed.

10 Studio command reference with the shipped example

Use this section if you skipped Codex or want to inspect Studio against a known, versioned result before generating anything. It intentionally collects the inspection, mock run, resume, model, and deployment controls using `zippergen/examples/tutorial_review.py`. If you completed the generated-workflow sections above and those controls are already clear, skip to Section 15.

From the project root, open Studio with the tutorial workflow already selected:

```
zippergen studio zippergen/examples/tutorial_review.py:tutorial_review
```

The beginning of the session looks like this; the absolute project path will of course be different on your machine:

```
ZipperGen Studio
Session context
Project    zippergen-tutorial
Root       /path/to/zippergen-tutorial
Workflow   OK zippergen/examples/tutorial_review.py:tutorial_review
Assistant  OK Codex CLI found
Next
  workflow show
zippergen [tutorial_review]>
```

The text in square brackets is the current workflow object name, not a deployment name. Enter `help` at any time. The public commands follow four main areas: `workflow` for specification, implementation, inspection, and validation; `model` for connections, configurations, checks, and assignments; `run` for development execution and its durable state; and `deploy` for installed operation. The important commands are:

```
project        # complete project inventory
project rename "NEW NAME" # logical name only; root stays unchanged
settings       # global learning, interpreter, assistant, editor, output
settings set learning off # applies to every local project
settings set assistant codex # bare workflow implement uses this
settings set editor micro # global terminal editor preference
settings set output banner # connected three-line command banner
current        # specification, workflow, models, run, deployment
editor show    # effective terminal editor and preference source
editor set micro # alias for the global editor setting
workflow       # workflow-development dashboard
workflow create # create/reopen specification.md
workflow refine # create/reopen the one pending refinement
workflow edit spec # edit the canonical specification
workflow edit code # edit the selected Python source
workflow files  # list entry module, local imports, and declared resources
workflow show source # inspect the authored entry module
workflow show spec # canonical requirements
workflow show pending # inspect the pending refinement
workflow show  # choose a semantic code view
workflow diff  # compare intent and semantics with the available baseline
```

```
workflow status # prepared, running, implemented, failed/interrupted
workflow status --details # include task IDs and internal record paths
workflow implement codex # or: workflow implement claude
workflow validate # technical structure and projection check
workflow discard # archive rejection; does not revert working-tree files
workflow history # specification and implementation history
workflow list   # discover workflow entry points; does not validate
workflow import /path/to/workflow.py # copy external source into this project
workflow select # select one by number, name, or complete reference
model         # show connections, configurations, and assignments
model setup   # guided provider, configuration, and assignment setup
model provider configure NAME # configure an API key or local endpoint
model provider check [NAME] # verify provider connectivity
model config create [NAME] # name one provider/model pair
model config check [NAME] # verify exact model availability
model assignments # show routes and cached last-check state
model assignments check # check only configurations used here
model assign PARTICIPANT NAME # route a participant to a configuration
model assign PARTICIPANT.ACTION NAME # override one LLM action
model config rename OLD NEW # rename it and migrate all assignments
connector # providers, configurations, assignments, and bindings
connector setup # guided setup for every connector used here
connector provider configure telegram # store the bot token privately
connector provider configure google # authorize Google Sheets privately
connector provider check [NAME] # check provider authentication
connector config create [NAME] # name a chat, sheet, or other resource
connector config check [NAME] # check the exact resource
connector assign PARTICIPANT NAME # route all human actions on a participant
connector assign PARTICIPANT.ACTION NAME # override one human action
connector assignments # show every effective human route
connector assignments check # check only configurations used here
run            # fresh guided, durable development run
run inspect [PARTICIPANT] [--watch] # current or live local position
run tasks      # pending decisions for the current development run
run approve    # review and answer a pending run decision
run trace      # recent events for the current development run
resume         # continue the current incomplete managed run
runs           # list this project's managed development runs
project reset  # choose fresh design, state-only reset, or cancel
deploy         # create or update a guided named deployment
deploy list    # list named deployments
deploy show [NAME] # bundle, service, run, and stable store
deploy inspect [NAME] [PARTICIPANT] [--watch] # durable local positions
deploy tasks [NAME] # pending decisions with complete context
deploy approve [NAME] # review and answer a pending decision
deploy trace [NAME] # recent durable deployment events
deploy storage [NAME] # sizes, event counts, and snapshot coverage
deploy storage compact [NAME] # safe stopped cleanup
deploy doctor # detailed readiness checks
deploy logs # recent service output
deploy logs reset [NAME] # archive and reset visible log history
deploy start # start the remembered installed service
deploy restart # restart it
deploy stop # stop it without deleting durable state
deploy remove [NAME] # unregister and archive one complete deployment
deploy remove NAME --purge # permanently delete its owned artifacts
exit           # leave Studio
```

These are Studio commands entered after the `zippergen [tutorial_review]>` prompt. Do not prefix each one with `zippergen` while already inside Studio.

10.1 Inspect the workflow as code

Enter `workflow show`. Studio displays:

```
1. Authored source
2. Overview
3. Protocol
4. Communications only
5. Actions and prompts
6. Complete workflow
7. One participant
8. Selected participants
```

Choose a number. Every result is code derived from the semantic workflow; the menu is only a convenient selector. It does not introduce a separate diagram format. You can also name a view directly:

```
workflow show overview
workflow show protocol
workflow show communications
workflow show actions
workflow show full
workflow show agent Reviewer
workflow show agents Writer Reviewer
```

`workflow show agent Reviewer` is the exact program produced by formal projection for Reviewer. `workflow show agents Writer Reviewer` is instead a focused global view that retains those participants and marks hidden peers as explicit boundaries. The distinction is explained fully later in the manual.

10.2 Run durably in the same terminal

Enter `run`. Studio validates first and asks for each declared workflow input, including its type and default:

```
zippergen [tutorial_review]> run
Workflow tutorial_review: valid
Request to answer (str) ["Explain why the sky is blue in two sentences."]:
Maximum revisions after the initial draft is rejected (int) [2]: 3
Run tutorial_review-<timestamp>

Draft:
[draft_reply:draft]

Automated reviewer notes:
[assess_draft:concerns]

Approve this draft? [y/n]: n

Draft:
[revise_reply:draft]

Automated reviewer notes:
```

```
[assess_draft:concerns]

Approve this draft? [y/n]: y
Result: [revise_reply:draft]
Next: show another view, start a new run, or prepare deployment.
```

Press Enter at the first input prompt to accept its displayed default. The mock LLM deliberately returns recognizable placeholders. Rejecting the first draft creates and presents the next durable human task in the same terminal; no second terminal and no `tasks` or `approve` command is required.

The run still uses SQLite. Studio creates a new, unique store and a small JSON run record below this project's managed workspace. Enter `current` to see the exact managed path, or `runs` to see earlier runs. Starting a new `run` never silently reuses an old result.

10.3 Return after a closed terminal or computer crash

The current workflow and run are remembered outside the Git checkout. Return to the same project root and reopen Studio:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
```

Then inspect and continue:

```
current
resume
```

`resume` reuses the recorded inputs and SQLite store. If a human task was pending, it is presented again in the terminal. Studio also compares the current workflow's semantic fingerprint with the one recorded when the run started. If the workflow meaning changed meanwhile, it preserves the old store and asks you either to restore matching source or start a new run.

No export needs to be repeated after the crash. By default, Studio uses `$HOME/.zippergen/workspaces`. The later reference exercise deliberately overrides `ZIPPERGEN_HOME` so that every low-level artifact is visible in one tutorial directory; that override is educational and optional, not a normal Studio prerequisite.

10.4 Start from a prompt, or refine an existing workflow

For a multiline description, let Studio open the canonical project document:

```
workflow create
```

For a short idea, provide the description on the same line:

```
workflow create Draft an answer and require human approval.
```

Studio saves a structured coding-assistant task. It asks for visible Python source, focused mock/fake tests, validation, communication and full views, exact participant projections, and no deployment. The stable generated mirror is `.zippergen/current-task.md`, which is ignored by Git; timestamped archives remain outside the checkout.

Current assistant boundary

Studio's `workflow implement codex` and `workflow implement claude` commands launch the selected local coding tool on the current implementation request. Neither is a ZipperGen workflow model provider, and neither automates deployment. The assistant edits ordinary files in the checkout; ZipperGen then loads, validates, projects, renders, and runs the result. Integrations can consume the same generated request directly. After source creation, enter `workflow list` and `workflow select` to discover and select the new workflow. The request is not a queued job: Studio records the synchronous assistant return as `implemented`, then prints the changed files, assistant checks, and assistant report. Use `workflow diff` and `workflow validate` for detailed inspection.

For an existing selected workflow, open or reopen the one pending change:

```
workflow refine
```

Studio owns both filenames, saves a semantic pre-edit baseline, and includes the canonical specification plus pending refinement in the assistant brief. The assistant is required to update `specification.md`, preserve behavior not mentioned in the request, validate the result, inspect changed projections, and run `zippergen diff` against that baseline. The pending request remains inspectable. `workflow discard` archives and clears it without reverting working-tree files.

10.5 Use a real LLM without repeating an API-key export

Remain on the mock backend until the complete local path works. First follow the account, billing, key-creation, and revocation precautions in the real-model readiness section. When ready, configure one model for Writer and another for Reviewer:

```
model provider configure openai
model config create drafting
model config check drafting
model assign Writer drafting
model provider configure anthropic
model config create careful-reviewer
model config check careful-reviewer
model assign Reviewer careful-reviewer
model
run
```

Studio remembers the named configurations and assignments per workflow. Provider setup collects each key first; configuration creation then asks for the exact model. Enter each requested key without echoing its value:

```
OPENAI_API_KEY:
Provider [openai]:
Model identifier [gpt-4o-mini]:
ANTHROPIC_API_KEY:
Provider [anthropic]:
Model identifier [claude-sonnet-4-6]:
```

The secrets are stored once below `$HOME/.zippergen/workspaces/<project>/`, in a file named `development.secrets.json` with owner-only mode-0600 permissions. Later runs and post-crash resumes reuse it. The values are not copied into `workspace.json`, a run record, an assistant brief, source code, or Git. Values already supplied by the process environment are used temporarily but are not copied into the private file.

This is lightweight local secret storage, not an operating-system keychain or enterprise secret manager. Deployment secrets are scoped separately. When a selected real provider has a matching key in private Studio storage, the first deployment explicitly offers to reuse it. Pressing Enter accepts the default; Studio transfers the hidden value directly into that deployment's own private mode-0600 secrets file. It never displays the value or puts it in project state. Answering `n` preserves the option to enter a distinct deployment credential. A later deployment with the same name keeps its already-saved deployment key without asking for it again.

11 Studio language, settings, and editors

These controls are useful after the first workflow succeeds, but none is a prerequisite for writing its specification. Studio's exact commands remain the durable, scriptable interface; ordinary language is a discoverability layer that always exposes the resulting command plan.

11.1 Commands and ordinary language

At the Studio prompt, requests such as the following use the deterministic interpreter and need no model, network, or API key:

```
What is the current state?
Show me the whole protocol.
Assign the mock model to Writer.
Please restart ZipperGen Studio.
```

They become `current`, `workflow show protocol`, `model assign Writer mock`, and `studio restart`. Studio prints the exact plan before its result. Starting or stopping execution, deployment operations, and destructive changes require confirmation. Use `plan TEXT` to preview an interpretation without executing it and `ask TEXT` to request interpretation and execution explicitly.

If unmatched declarative or imperative prose might describe the application, Studio offers to treat it as a workflow requirement. Before `specification.md` exists it proposes `workflow create`; afterward it proposes `workflow refine`. The offer does not depend on framework words such as "workflow" or "participant". Enter `y` to store the requirement, `n` to cancel, or `command` to interpret the same sentence as an operation without changing the specification. Questions and recognizable operational or troubleshooting requests remain on the command path. Explicit `ask TEXT` and `plan TEXT` bypass the requirement offer.

Short phrases use project context: `run it` means the selected workflow and `stop it` means the remembered deployment. Ambiguous `start over` asks whether the intention is a new run, discarded refinement, Studio restart, or fresh project design. The unambiguous `reset everything` proposes the recoverable fresh-design reset.

When repository details are required and no built-in or learned interpretation matches, Studio may run an already authenticated Codex or Claude Code CLI once. It receives non-secret, read-only project context and may return only a structured plan made from documented Studio commands. Studio validates and executes the plan itself; model text is never run as shell code.

11.2 Global settings and project-local learning

Inspect and change the language layer with:

```
settings
```

```
settings set learning on
settings set interpreter auto
settings set assistant codex
settings set editor micro
settings set output banner
language
language set auto
language set codex       # or: language set claude
language history
language learned
language forget L001
```

`settings` contains owner-private preferences shared by every local ZipperGen project: learning policy, interpreter, default coding assistant, editor, and output style. It is stored below `$ZIPPERGEN_HOME/settings.json`. Learning policy is global, but learned interpretations and command history are project-local, so one project cannot silently teach another.

`auto` prefers Codex and then Claude for a repository-aware fallback and reuses the CLI's own login; no ZipperGen model-provider key is involved. `language set off` retains deterministic and already learned interpretations without launching a CLI. After a successful CLI plan, Studio stores the accepted request, structured commands, and outcome in owner-private project state and can generalize recognized values such as participant names. Use `settings set learning off` to stop adding entries globally, or `language forget ID|all` to remove entries from the current project.

Raw CLI output is not learned. Secret-looking requests are rejected before interpretation and are not stored. Provider credentials belong in the private prompt opened by `model provider configure`.

11.3 Automatic and explicit terminal editors

Studio checks an explicit preference, then `$VISUAL` and `$EDITOR`, then installed `micro`, `nano`, `vim`, and `vi`. Inspect the result or save a global preference:

```
editor show
editor set micro
editor show
editor reset
```

`editor set nano` is equally valid and aliases `settings set editor nano`. The choice is machine-specific Studio state, not workflow source, and survives terminal closure and reboot. `editor reset` restores automatic discovery.

An editor temporarily takes over the same terminal. Save and exit to return to the unchanged Studio session. Studio launches the executable directly; it does not invoke an LLM, shell script, provider, or MCP server. Override the editor for one operation without changing the saved preference:

```
workflow edit code --editor nano
workflow create --editor micro
```

For a command with arguments, quote the complete value, for example `--editor "code --wait"`. Studio accepts edited files only inside the project root and stops when the editor reports failure.

12 The development-to-deployment boundary

Deployment may install packages, run declared setup such as OAuth, create a source bundle and managed environment, and start a macOS or Linux user service. It is therefore always a separate, explicit command. With the tutorial still selected, return to mock unless a real deployment is intentional, then choose a separate name:

```
model default mock
model inherit Writer
model inherit Reviewer
deploy tutorial-review-studio --no-start
```

The same guided deployer described later carries the current default and participant and action model routes, collects declared settings and deployment-scoped secrets, validates readiness, and prints the resulting deployment name. If a selected provider key is already configured, Studio first shows only its environment-variable name and asks whether to reuse it; accepting avoids another secret-entry prompt while retaining a separate deployment copy. The `--no-start` flag prepares the bundle and deployment environment without installing or starting a user service. After a successful preparation, Studio remembers that name. Inspect it first:

```
deploy doctor
deploy show
```

The deployment store is already initialized and should be reported as ready with no run data. Warnings about a missing first-run log and an uninstalled service are expected at this point; a doctor failure is not. Only when service startup is intentional and authorized, enter:

```
deploy start
deploy show
deploy tasks
deploy logs
deploy restart
deploy show
deploy stop
```

`deploy start` is the first command in this sequence that changes the user service manager. `deploy logs` returns after showing recent output; it does not stop the service. `deploy stop` leaves the deployment artifacts and durable store in place. To retire the complete deployment safely, use:

```
deploy remove tutorial-review-studio
```

Studio previews the service state and exact owned paths before it changes anything. The default form stops and unregisters the service. It then moves the profile, private secrets, generated launch files, bundles, managed environment, stable store, SQLite sidecars, and log into a private archive below `ZIPPERGEN_HOME/trash/deployments`. The deployment disappears from `deploy list`. Project source, specifications, implementation history, shared model configurations, and shared connector configurations remain. The archive preserves the files for manual recovery. Studio does not currently provide an automatic restore command.

For deliberate permanent cleanup, use `deploy remove tutorial-review-studio --purge`. This form requires an explicit deployment name. Without `--yes`, Studio also requires you to type that exact name. There is no remove-all form. Permanent purge has no recovery archive.

You may also provide a deployment name explicitly:

```
deploy show tutorial-review-studio
```

The scriptable equivalents outside Studio remain `zippergen status tutorial-review-studio` and so on.

Deployment is a real operational action

Do not enter `deploy`, `deploy start`, or `deploy restart` merely to explore the menu. Read the deployment chapters first, remain on mock/fake services, and understand the declared setup and effects. The `workflow show full` view exposes prompts and deployment requirements before this transition.

13 Moving a workflow to a server

`deploy` installs a service on the machine Studio is running on. There is no remote deployment command. A server deployment therefore begins with a step that is easy to overlook: the workflow source has to reach the server, and Studio has to run there. This section covers that step. Everything earlier in this chapter then applies unchanged on the server itself.

13.1 Clone for ongoing work, import for a single file

Two arrangements work. Prefer the first for any workflow you will change again.

Keep the workflow in a repository, clone it on the server, and use the clone as the project root. Updates are then an ordinary `git pull`, and no import step is required at all:

```
git clone git@github.com:you/your-workflows.git ~/workflows
cd ~/workflows
zippergen studio
```

Use `workflow import` when you want to pull one external file into an existing project — a shipped example, or a workflow kept outside the repository:

```
scp -r ./call-intake user@server:~/staging/
```

Then, in Studio on the server:

```
workflow import ~/staging/call-intake/call_intake.py:call_intake
```

13.2 Copy the directory, not only the entry file

A workflow may name local files in its declarations: connector configuration, prompt text, or data an action reads. `workflow import` reads the source and copies those files together with the entry module. A manual `scp` of one `.py` file does not. When you copy by hand, copy the directory that contains the workflow rather than the file alone.

13.3 Credentials stay on the machine that created them

Copying a project directory moves no secret. Provider keys, connector tokens, and Google credentials are stored below `ZIPPERGEN_HOME/workspaces` with owner-only permissions, outside the project directory. A project directory can therefore be copied, pushed to a repository, or handed to somebody else without carrying credentials with it.

Model and connector assignments are held in the same private workspace state, so they do not travel either. After the source arrives, configure them on the server exactly as you did locally:

```
model setup
connector setup
```

This is expected rather than a missing step. Each machine holds its own credentials, and a workflow file that reached the server by any route cannot bring another machine's secrets with it.

The implementation request is also machine-local in the current Studio layout. Source transfer therefore does not carry its fingerprint. On a fresh server, Studio states that the record is unavailable and proceeds only after technical validation. You may run `workflow implement` there when you want the server to establish its own local coherence record, but deployment does not require this temporary workaround.

13.4 Validate on the server before deploying

Run the ordinary inspection commands once on the server before the first deployment. They load the module in that environment and fail early if a resource file did not arrive or an import does not resolve there:

```
workflow validate
workflow show
```

Only then continue with model and connector configuration and, finally, `deploy`.

The same Studio, different commands

Studio behaves identically on both machines. Authoring commands such as `ask`, `plan`, `workflow refine`, and `workflow implement` use a coding assistant. Validation, running, deployment, and inspection do not invoke that assistant. A first Studio deployment on a fresh server warns that no local implementation record is available, then continues through the normal technical validation and readiness checks.

---

14 Studio and scriptable CLI equivalents

The rest of this manual intentionally exposes the lower-level operations. Use this table to understand why those sections contain more commands.

Need	Recommended Studio path	Detailed/scriptable path
Select workflow	`workflow list`, then `workflow select`; remembered per project	Supply `path.py:workflow` to each command.
Inspect	`workflow show` selector	`zippergen show` with `--detail`, `--communications`, or participant flags.
Develop	`run`; guided inputs and unique managed store	`zippergen run` with explicit inputs, timeout, and optionally store.
Human decision	Inline in the same terminal	`tasks` and `approve` from another terminal or notification adapter.
Continue	`resume`	Re-run against the same explicit SQLite store.
Inspect durable state	`run inspect`, `run tasks`, `run approve`, `run trace`, or the corresponding `deploy` commands	Pass the exact `--store` path to status/task/trace commands.
Specification change	`workflow create` or `workflow refine`	Manually request a snapshot, assistant edit, validation, views, projections, and semantic diff.
Deploy/operate	`deploy`, then `deploy show`, `deploy tasks`, `deploy approve`, `deploy trace`, `deploy logs`, `deploy restart`, `deploy stop`, and `deploy remove`	Named deployment commands, explicit options, profiles, and service-manager operations.

The topical Studio chapters should be enough to make the workflow concrete. The detailed reference that follows makes each operation auditable and automatable.

15 How to use the detailed reference

The preceding topical chapters describe the recommended practical cycle. The remaining reference chapters unpack its machinery so that commands can be audited, automated, and debugged. They explain how to perform all of the following explicitly:

1. Install an isolated Python environment for ZipperGen.
2. Open the running example in Studio and discover the available operations without memorizing flags.
3. Run a three-participant workflow with the built-in mock LLM.
4. Read the global protocol, communication-only view, selected-agent view, and exact single-agent projections in the terminal.
5. Run the workflow durably through SQLite and approve a human task from a second terminal.
6. Ask a coding assistant to refine a copy of the workflow.
7. Compare the old and new meanings with a semantic snapshot and diff.
8. Create a named deployment with a source bundle and managed Python environment.
9. Run that deployment in the foreground, then optionally as a supervised macOS or Linux user service.
10. Diagnose, inspect, restart, reconfigure, redeploy, and stop it.

The first pass uses the mock LLM. Studio writes managed development state below `$HOME/.zippergen/workspaces` by default. The later transparent/reference exercise chooses a separate `tutorial-runtime` directory so its profiles, stores, logs, and bundles are especially easy to inspect. No API key and no external service account are required for either mock path.

16 Conventions and safety

16.1 How to read commands

All shell commands appear in boxes such as this:

```
zippergen --help
```

Do not type a leading dollar sign if another document uses one to represent the shell prompt. Run commands one at a time and read their output before moving on. Unless a section says otherwise, run commands from the tutorial project root— the directory containing `zippergen.toml`, `specification.md`, `workflows`, and the nested `zippergen` framework checkout.

Text such as ⟨*TASK-ID*⟩ is a placeholder. Replace the complete placeholder, including angle brackets, with the value printed on your machine.

There are three places where text is entered:

Place	Prompt usually visible	Examples
Operating-system shell	`%`, `$`, or a directory name	`cd`, `git`, `zippergen`, `codex`
ZipperGen Studio	`zippergen [workflow]>`	`workflow show`, `model`, `run`, `deploy`, `deploy show`
Coding assistant	Codex conversation prompt	The Studio-generated task; `workflow implement` supplies it automatically

Do not enter a Studio command directly into zsh, and do not prefix a Studio command with `zippergen` while Studio is already open.

Long shell commands use a backslash to continue on the next physical line. The backslash must be the final character on its line—no spaces or comments may follow it. Paste the complete listing, or run it as one line after removing the backslashes and line breaks. Errors such as `zsh: command not found: --input` mean the shell ended the command too early; re-enter the complete command with intact continuations. A typographic dash such as "–" copied from prose is not two ASCII hyphens; command options must begin with two ordinary `-` characters.

16.2 When two terminal windows are used

Studio presents development approvals in one terminal. The later low-level and external-adapter examples deliberately use two terminals:

- **Terminal A** runs a workflow and waits.
- **Terminal B** checks status and approves or rejects the task.

For those later examples, both terminals must be in the project root and must set the same `ZIPPERGEN_HOME` value when the manual asks for it. This does not apply to the opening Studio path.

16.3 Optional transparent CLI workspace

The recommended Studio path needs no runtime export. The later low-level exercise deliberately makes profiles, stores, logs, and bundles visible. For that exercise, use one directory as both tutorial root and Git repository, with runtime data in an ignored subdirectory:

```
$HOME/Projects/zippergen-tutorial/  # project Git root and Codex workspace
|-- .git/
|-- zippergen.toml
|-- specification.md
|-- workflows/
|-- tests/
|-- zippergen/          # separately cloned framework, ignored by project Git
`-- tutorial-runtime/  # ignored profiles, stores, logs, and bundles
```

The `Projects` directory is a convenient convention, not a ZipperGen requirement. This layout avoids putting a Git checkout directly in `$HOME` and makes the entire tutorial easy to find. The lowercase, hyphenated directory name also avoids the extra quoting and escaping that spaces require in shell commands. Project initialization makes `.gitignore` exclude both the nested framework checkout and `tutorial-runtime`, so neither can be committed accidentally. Normal Studio use and deployments use `$HOME/.zippergen`. The detailed reference exercise instead sets an explicit, visible tutorial location:

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
```

Run that export again in every new terminal used for the later reference exercise. Studio does not require it. Do not change it to `$HOME`, `/`, or another broad directory.

Start with the mock backend

Do not configure a paid model or live external service until the complete mock cycle works. The mock backend returns recognizable placeholders such as `[draft_reply:draft]`; this is expected and proves the coordination path without network calls or charges.

17 Assistant actions and connector boundaries

17.1 Coding assistants inside an executing workflow

The Studio command `workflow implement codex` is a development command: it asks an assistant to edit the current project. The `@assistant` decorator is different. It declares repository-aware assistant work as an explicit action inside the protocol. The owning lifeline, typed inputs, static instruction, result, trace event, and semantic change therefore remain visible.

```
from zippergen import Lifeline, assistant, workflow

Maintainer = Lifeline("Maintainer")

@assistant(
    instructions_file="prompts/update_release_notes.md",
    access="write",
    external_tools="none",
    shell="restricted",
    workspace=".",
)
def update_release_notes(change: str) -> str: ...
```

```
13
14 @workflow
15 def maintain(change: str @ Maintainer) -> str:
16     Maintainer: report = update_release_notes(change)
17     return report @ Maintainer
```

Use exactly one of `instructions=` and `instructions_file=`. A Markdown path is relative to the project, fingerprinted by semantic inspection, checked by `validate`, and copied automatically into a guided deployment bundle. Function parameters are dynamic workflow data and are passed separately from that static instruction.

```
1 zippergen run workflows/maintain.py:maintain \
2   --assistant codex \
3   --input 'change=Document the new retry policy.'
```

`--assistant claude` selects Claude Code instead. Python callers may select a CLI or inject a custom backend:

```
1 workflow.configure(assistant="codex")
2 workflow.configure(assistant_backend=my_backend)
```

A Studio run uses `run --assistant codex`; that selection is recorded so `resume` retains it. A recorded SQLite result is replayed without launching the assistant again. Nevertheless, make the requested file operation safe to resume: a crash can occur after files change but before the action result is recorded.

Access is part of the action's reviewed semantics. The safe default is `access="read-only"`; use `access="write"` explicitly for an implementation action. ZipperGen maps the policy to the selected CLI's non-interactive sandbox or permission mode; a prompt that merely asks a reviewer not to edit is not the security boundary.

Filesystem access and external-tool access are separate capabilities. The default `external_tools="none"` disables configured MCP servers, dedicated web tools, assistant subagents, and comparable integrations. Use `external_tools="configured"` only when the reviewed action intentionally needs the user's configured tools.

Shell capability is a third reviewed policy. The default `shell="restricted"` selects the strongest practical boundary offered by each backend:

- Codex retains command execution inside its read-only or workspace-write sandbox. When external tools are disabled, network access is disabled structurally as well. Strict configuration parsing makes an unsupported isolation key fail the action instead of being ignored.
- Claude receives no Bash tool. Setting `shell="enabled"` permits Bash, but Claude does not then provide Codex's structural network isolation; `validate` displays a warning.

For a provider-independent hard boundary, let the assistant edit without a shell and perform a predetermined check in a subsequent visible `@effect`. The effect should contain a fixed executable and fixed arguments; never execute a command string returned by the assistant. Arbitrary shell access with provider-independent network isolation requires an external operating-system or container sandbox.

Access, external-tool policy, and shell policy always appear in semantic snapshots and diffs. Validation also warns when a write workspace contains the executing workflow. That workspace permits self-modification, so the static instruction must protect the running workflow and must not silently authorize deployment, service control, commits, pushes, or unrelated external mutations.

The shipped `examples/codex_claude_review.py` workflow demonstrates a bounded Codex–Claude loop: Codex owns every repository change, Claude reviews read-only after each candidate, success requires Claude approval, and Codex produces the final read-only assessment.

17.2 Connector providers, configurations, and assignments

Human authority remains visible in workflow code as an `@human` action. Its delivery mechanism does not belong in that code. Studio discovers the owning participant and supports the same three-stage lifecycle as models:

```
connector provider configure telegram
connector provider check telegram
connector config create telegram-approvals
connector config check telegram-approvals
connector assign HumanApprover telegram-approvals
connector assignments
connector assignments check
```

The provider owns private authentication, such as a Telegram bot token. A named configuration chooses a reusable resource, such as one Telegram chat. An assignment routes compatible actions to that configuration. The token is stored outside the repository. Deployment copies only routing metadata into the ordinary profile and places the credential in its private secret file.

A participant assignment covers every human action on that lifeline:

```
connector assign HumanApprover telegram-approvals
```

An exact action may override it:

```
connector assign HumanApprover.approve_contract legal-approvals
connector inherit HumanApprover.approve_contract
```

Several participants may share one configuration. Sharing a bot and chat does not serialize their workflow steps. Every durable human task has an opaque, one-time token. Telegram callback buttons include that token and the selected option. A delayed, duplicate, or out-of-order external message therefore cannot be mistaken for the next FIFO message. A numbered text answer is accepted only when its task is identified by the token or by a direct reply to the original Telegram message.

The deployed service starts its connector bridge automatically:

```
deploy tasks
deploy show
```

No manual notification synchronization command is required. The bridge watches the deployment's unique store, sends unseen tasks, consumes replies, and completes exactly the corresponding durable action. A development run starts the same bridge automatically when the selected workflow has an external human assignment. Without such an assignment, development keeps the human decision in the Studio terminal.

Non-human services remain explicit workflow capabilities. Gmail and Google Sheets use the same provider and configuration surface. Docs and Calendar can follow the same pattern. A workflow declares a logical requirement and binds that requirement to a saved resource configuration. Human delivery continues to use participant and action assignments.

17.3 Google Sheets resources

Google support is optional so that a basic ZipperGen installation remains small. In a source checkout, install it once with:

```
uv sync --extra google
```

For an installed package, use `pip install "zippergen[google]"`. Studio reports this exact command if the support is missing. A managed deployment whose workflow declares Gmail or Google Sheets installs the optional support automatically.

The workflow owns the meaning of a table operation. It declares the logical table, columns, stable key, and whether an effect reads or writes data. It does not contain a spreadsheet ID or OAuth token:

> Records owns a project-records table with record_id, title, and status columns. One effect upserts a JSON record by record_id. Another effect reads all rows into a JSON workflow variable. Both effects use a required read-write Google Sheets connector. Keep the spreadsheet and Google account as deployment configuration.

A repository-aware assistant can translate that requirement into the visible workflow declaration and effects below:

```python
import json

from zippergen import (
    ConnectorRequirement,
    Json,
    effect,
    read_json_rows,
    upsert_json_row,
)

zippergen_connectors = (
    ConnectorRequirement(
        name="project-records",
        kind="google-sheets",
        participant="Records",
        capabilities=("read-rows", "upsert-row"),
        access="read-write",
    ),
)

COLUMNS = ("record_id", "title", "status")

@effect(
    connector="project-records",
    operation="upsert-json-row",
)
def write_record(record: Json) -> str:
    return upsert_json_row(
        "project-records",
        json.dumps(record, sort_keys=True),
        columns=COLUMNS,
        key_field="record_id",
    )

```

```
@effect(
    connector="project-records",
    operation="read-json-rows",
)
def read_records() -> Json:
    return json.loads(
        read_json_rows(
            "project-records",
            columns=COLUMNS,
        )
    )
```

The stable key is important. Durable execution can retry an effect after a process failure. A blind append could create the same row twice. The upsert checks the key and replaces the existing row when needed.

The `Json` coordination type keeps the record structured inside the workflow. It accepts ordinary Python values made from `None`, booleans, numbers, strings, lists, and dictionaries with string keys. ZipperGen validates the full nested value before durable execution and preserves it across messages, crashes, and resume. The Google helper still receives JSON text at the external API boundary. Arbitrary Python objects and pickle data are not accepted as workflow variables. Lists and dictionaries must be ordinary built-in containers, not subclasses.

The user configures the environment after selecting the workflow:

```
connector setup
```

Google requires one account-level preparation before Studio can authorize the connector:

1. Create or select a project in Google Cloud Console.
2. Enable the Google Sheets API for that project.
3. Configure the OAuth consent screen for the intended test users or organization.
4. Create an OAuth client with application type *Desktop app*.
5. Download its JSON file. Keep it outside the project repository.

For a workflow with one Google Sheets requirement, guided setup performs these steps:

1. On a computer with a browser, it asks for the OAuth Desktop app JSON path and opens Google authorization directly.
2. In an interactive SSH session without a browser, it prints one `uvx` command. Run that command on your own computer, not on the server. It needs no local ZipperGen installation. Copy the whole `zg-google-v1.` line it produces into Studio's hidden prompt. No file upload or SSH tunnel is needed.
3. It asks for a spreadsheet URL or ID and a tab name.
4. It checks that the spreadsheet and tab are reachable.
5. It binds the saved configuration to the logical requirement.

The equivalent explicit commands are:

```
connector provider configure google
connector provider check google
connector config create project-records
```

```
connector config check project-records
connector bind project-records project-records
connector assignments check
```

The provider stores the authorization privately. It does not store the OAuth client file or retain a user-managed client-file path. The configuration stores only the spreadsheet ID and tab. The workflow binding stores only the configuration name. During deployment, ZipperGen copies the resource metadata into the immutable deployment profile and copies the OAuth credential into the deployment's private secret file.

Studio derives Google scopes from each connector's declared `access`. A read-only Sheets requirement requests `spreadsheets.readonly`. A write or read-write requirement requests `spreadsheets`. Google does not provide a write-only Sheets scope for an arbitrary existing spreadsheet. The broader scope applies to the account's spreadsheet files, not only to the configured tab. Use a dedicated account, a dedicated spreadsheet, or protected ranges when a narrower operational boundary is required. Studio records the scopes that Google actually granted. Connector checks, runs, and deployments stop early if those scopes no longer cover the selected workflow.

17.4 Gmail and Sheets in one workflow

The call-intake example declares two connector requirements. Mailbox owns a `gmail` requirement for reading messages, marking them processed, and creating or sending replies. Table owns a `google-sheets` requirement for reading and updating the call register. The source does not contain a Gmail account, OAuth token, search query, spreadsheet ID, or tab:

Before opening Studio:

1. Enable the Gmail API and Google Sheets API in one Google Cloud project.
2. Configure the OAuth consent screen.
3. For an external test application, add the Gmail account as a test user.
4. Create a Desktop app OAuth client and download its JSON file.

Publishing an external application with Gmail access may require additional Google review. Use a dedicated test account while developing.

```
workflow select examples/call_intake.py:call_intake
workflow show communications
workflow validate
model setup
connector setup
connector assignments check
```

For this workflow, `connector setup` performs one Google browser authorization with the Gmail and Sheets scopes required by the declared operations. It then creates and checks two named configurations:

- a Gmail configuration containing account `me` and a search query such as `is:unread in:inbox to:calls@example.com`;
- a Google Sheets configuration containing a spreadsheet ID and tab.

Both configurations may use the same Google provider authorization. They are still separate resources and may be rebound independently.

If Studio runs on a server without a browser, copy the displayed authorization command to your own computer, not to another server terminal. `uvx` fetches the helper without a local ZipperGen installation. The command asks for the downloaded Desktop app JSON, opens Google, and prints one long line starting with `zg-google-v1.`. Copy that whole line into Studio's hidden prompt. The server needs no browser, client file, callback tunnel, or direct connection from your own computer.

A read-only Gmail requirement requests `gmail.readonly`. A write or read-write Gmail requirement requests `gmail.modify`. The call-intake workflow declares read-write access because it reads messages, marks them processed, and creates drafts or sends replies.

The deployment fields ask only for application policy such as certified senders, the expected intake address, reply mode, rate limit, and polling interval. The default reply mode is `draft`. Inspect the drafts in a dedicated test account before choosing `send`.

```
workflow diff
workflow validate
deploy call-intake
deploy show call-intake
deploy logs call-intake
```

Deployment takes an immutable snapshot of both connector bindings and copies the OAuth credential into the deployment's private secret file. Readiness checks verify the Gmail account and the selected spreadsheet tab before the service starts. The deployed service owns one durable store, so a crash can resume the protocol without losing which message or table operation was in progress.

18 Reference: prerequisites and installation details

This guide targets macOS and Linux. ZipperGen requires Python 3.11 or later. The commands use `uv` to manage Python and the local environment. If the opening setup already ended with a working `zippergen --help`, do not clone or install again; use this section only to understand or repair the setup.

18.1 Check what is already installed

```
git --version
curl --version
python3 --version
uv --version
```

If `git` is missing, install the standard Git package for your operating system. If `uv` is missing, use the official macOS/Linux installer:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

The installer may ask you to restart the terminal so its binary is on `PATH`. The current official alternatives, including Homebrew and `pipx`, are documented at https://docs.astral.sh/uv/getting-started/installation/.

18.2 Clone and install ZipperGen

For a new checkout:

```
mkdir -p "$HOME/Projects"
cd "$HOME/Projects"
mkdir zippergen-tutorial
cd zippergen-tutorial
git init
git clone https://github.com/zippergen-io/zippergen.git zippergen
uv python install 3.12
uv sync --project zippergen --python 3.12
uv tool install --force --editable ./zippergen
zippergen --help
```

`uv sync` creates or updates the nested checkout's local `zippergen/.venv`. The editable tool installation exposes the short CLI while continuing to use that source. The parent project has its own Git history for the specification, workflows, and tests. All files remain on disk after closing a terminal or rebooting. Normally, returning to Studio requires only:

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
```

The current directory belongs to a shell session, so repeat the `cd` in each new terminal. You do *not* normally repeat `git clone`, `uv python install`, `uv sync`, `uv tool install`, or `codex login` after a crash or reboot. The optional low-level reference track later introduces `ZIPPERGEN_HOME` only to keep all runtime artifacts visible in one tutorial directory; Studio does not need that export.

Updating the checkout is a separate, occasional operation:

```
git -C zippergen status --short
git -C zippergen pull --ff-only
uv sync --project zippergen
```

If you deliberately use an existing checkout elsewhere, replace both paths with suitable locations in that checkout, or choose another isolated runtime directory and ensure Git ignores it.

Do not run `git pull` while you have changes you do not understand. First inspect `git -C zippergen status --short`; commit, stash, or ask for help when needed. Repeat `uv sync --project zippergen` after pulling dependency changes or if the local framework environment was removed.

18.3 Install Codex or Claude Code for Studio

The nested framework checkout ships the workflow skill below, while `zippergen/AGENTS.md` supplies its repository guidance:

```
zippergen/.agents/skills/zippergen-workflows/SKILL.md
```

Studio records the nested checkout in `zippergen.toml` and places these exact paths in every assistant task. See the official Codex skill documentation.

If the Codex CLI is not installed and Node.js/npm is available:

```
npm install --global @openai/codex
codex login
```

Complete the browser sign-in once. In ordinary manual use, Codex can be started in the parent tutorial project with:

```
cd "$HOME/Projects/zippergen-tutorial"
codex
```

Codex now sees the complete application project: its manifest, canonical specification, workflow code, tests, and nested framework checkout. Because the skill is in a nested repository, it need not appear in the parent's `/skills` list; the generated brief explicitly instructs Codex to read and follow it. The CLI and IDE extension are recommended local repository surfaces; see https://learn.chatgpt.com/docs/codex/cli.

Claude Code is an equal alternative. If it is not installed and Node.js/npm is available, install it and start it once to complete Anthropic's authentication:

```
npm install --global @anthropic-ai/claude-code
cd "$HOME/Projects/zippergen-tutorial"
claude
```

See Anthropic's Claude Code setup guide and CLI reference.

Normally, stay in Studio and enter `workflow implement codex` or `workflow implement claude`. Studio supplies the project directory and `.zippergen/current-task.md` instruction. Both tools use one-shot agent execution, perform the task, print their report, and return to Studio automatically instead of leaving an open assistant prompt. Codex retains the explicit `workflow implement codex --interactive` mode. Commands remain subject to each tool's configured permission rules. Neither path requires MCP. Any MCP server already configured in the chosen tool remains optional and is not configured or required by ZipperGen.

Coding-assistant authentication and a workflow's LLM API key are separate matters. The mock tutorial needs no workflow API key, and ZipperGen's provider configuration configures neither Codex nor Claude Code.

19 The running example

The example is `zippergen/examples/tutorial_review.py`. It has three lifelines:

- **Requester** initially owns a request and the maximum number of revision attempts, and receives the answer.
- **Writer** drafts a reply and revises it after rejection.
- **Reviewer** uses a separate LLM action to assess every draft, presents the draft and notes to a human authority, and owns the bounded retry loop.

The protocol is:

1. Requester validates `max_retries`, sends the request to Writer, and sends the retry budget to Reviewer.
2. Writer uses an LLM action to create a draft.
3. Writer sends the draft to Reviewer.
4. Reviewer uses an LLM action to identify concerns, then presents both the draft and concerns in a human confirmation action.
5. If Reviewer approves, the draft goes to Requester.

6. If Reviewer rejects while revision attempts remain, Reviewer returns the draft to Writer; Writer revises it and presents the new draft to Reviewer again.
7. If the retry budget is exhausted without approval, Requester receives an explicit failure notice instead of an unapproved draft.

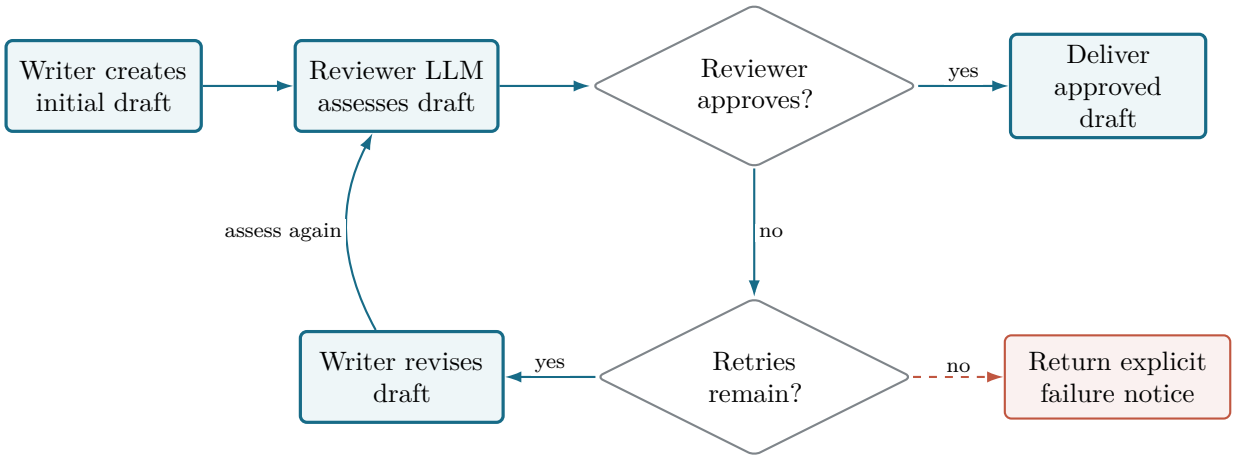

Figure 4: A rejection produces another reviewed draft only while the bounded retry budget remains. No unapproved draft is silently released.

What `max_retries` counts

The initial draft is not a retry. With `max_retries=2`, Writer may make at most two revisions after rejection, so Reviewer sees at most three human tasks: the initial draft and two revised drafts.

The complete, executable source is included in Appendix B.

19.1 Why the deployment declaration matters

The module also defines `zippergen_deployment`. It tells the guided deployer to collect:

- the LLM setting, defaulting to `mock`;
- the request text as a workflow input;
- the maximum number of revisions as a second workflow input, defaulting to `2`;
- an OpenAI API key only if the default or any assigned model starts with `openai`; and
- an Anthropic API key only if the default or any assigned model starts with `anthropic` or `claude`.

The declaration is data-only. A coding assistant can generate it, and `show --detail full`, `validate`, `doctor`, and `deploy` can inspect it without bespoke shell scripts.

20 Optional reference: one-process in-memory execution

Studio's durable `run` is the recommended development command. This short direct run is useful only for understanding the simpler in-memory backend or debugging the Python module itself. Run the example directly:

```
uv run --project zippergen python zippergen/examples/tutorial_review.py
```

Expected interaction:

```
Draft:
[draft_reply:draft]

Automated reviewer notes:
[assess_draft:concerns]

Approve this draft? [y/n]: y
Result: [draft_reply:draft]
```

Run it again and enter `n`. Writer produces `[revise_reply:draft]`, Reviewer produces a new `[assess_draft:concerns]`, and the human is asked again; enter `y` to accept that revision. If you keep entering `n`, the program stops after two revision attempts and returns an explicit failure notice. This direct program uses in-memory execution and an in-terminal human backend. It is convenient for the first smoke test, but it does not demonstrate durable restart behavior.

Internally, this memory backend runs entirely inside the one Python process started by `uv run --project zippergen python`. It creates one Python thread for each lifeline—Requester, Writer, and Reviewer in this example—and uses thread-safe FIFO queues, keyed by sender, receiver, and channel, for their messages. It does not create a separate operating-system process for each lifeline. When the program exits or crashes, the threads, queues, messages, and progress disappear together. The later SQLite backend still runs local lifelines in threads, but records communication and execution progress in the database so it can recover them after a restart.

21 Optional reference: inspect with scriptable CLI commands

Set the workflow spec once in your head: `zippergen/examples/tutorial_review.py:tutorial_review`. Read it as follows:

- Before the colon, `zippergen/examples/tutorial_review.py` is the Python file to load.
- After the colon, `tutorial_review` is the existing Python object to select. It comes from `@workflow def tutorial_review(...)` inside that file.

The suffix does not assign a new name; it locates the workflow object. The deployment name is separate: this example declares `tutorial-review` in `zippergen_deployment`, and deployment commands may supply it with `--name tutorial-review`. The underscore in the Python object name and the hyphen in the deployment name are intentional. The validation JSON makes the distinction visible as `"workflow": "tutorial_review"` versus `"deployment": {"name": "tutorial-review", ...}`.

The CLI currently expects the workflow spec as a positional argument, so the following commands repeat it for clarity.

21.1 Validate before running or deploying

```
zippergen validate zippergen/examples/tutorial_review.py:tutorial_review
```

Successful validation reports three unique lifelines, successful projections for Requester, Writer, and Reviewer, a valid deployment declaration, and a successful *canonical rendering* check. Here,

canonical rendering means that ZipperGen can turn the loaded workflow into its standard, normalized code views: the global protocol and the generated local view for each participant. It does not execute or deploy the workflow. Treat a validation failure as a blocker.

For machine-readable output:

```
zippergen validate zippergen/examples/tutorial_review.py:tutorial_review --json
```

21.2 Global detail levels and the communication-only view

The plain `show` command uses `--detail protocol` by default, so the first two protocol commands below are equivalent:

```
zippergen show zippergen/examples/tutorial_review.py:tutorial_review
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail
    protocol
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail
    overview
zippergen show zippergen/examples/tutorial_review.py:tutorial_review \
  --detail protocol --communications
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail
    actions
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail full
```

The four detail levels are:

- `overview`: names the workflow, lifelines, inputs, outputs, actions, and deployment counts, but replaces the workflow body with an ellipsis.
- `protocol`: shows the complete global workflow body, including communications, decisions, loops, parallel regions, and action calls. This is the default.
- `actions`: adds the action declarations and signatures; detailed LLM and planner prompt bodies remain hidden.
- `full`: adds prompts, available implementations, and the normalized deployment declaration.

The separate `--communications` filter keeps messages, decisions, and the return value but omits local action calls. It can be combined with a detail level when useful. Because `protocol` is the default, `--communications` alone is shorthand for `--detail protocol --communications`; this manual spells out the full form where the view is introduced.

21.3 One participant or a selected group

Exact single-agent projections:

```
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --agent
    Requester
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --agent Writer
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --agent
    Reviewer
```

A focused multi-agent view keeps hidden peers as explicit external boundaries:

```
zippergen show zippergen/examples/tutorial_review.py:tutorial_review \
  --agents Writer,Reviewer
```

Do not confuse these modes: `--agent Reviewer` is the exact formal local projection; `--agents Writer,Reviewer` is a global focus view.

Structured output for tools and coding assistants is also available:

```
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --format json
```

22 Optional reference: explicit durable store and two terminals

Studio already provides this behavior with a managed store and inline human decision. This lower-level pass is useful for automation, notification adapters, or understanding the exact SQLite/task commands. The process in Terminal A waits without losing its state; Terminal B finds and completes the human task.

22.1 Task, task ID, and token

The *task* is the pending work itself: in this example, the rendered draft and the request for a human decision. The *task ID* is its stable internal name. A trusted operator working locally can use that ID to select the task. A *token* is a separate, random bearer credential that an external adapter can use instead of trusting a raw task ID. Protect a token like an approval link. Both point to the same task; neither is the approval or rejection decision.

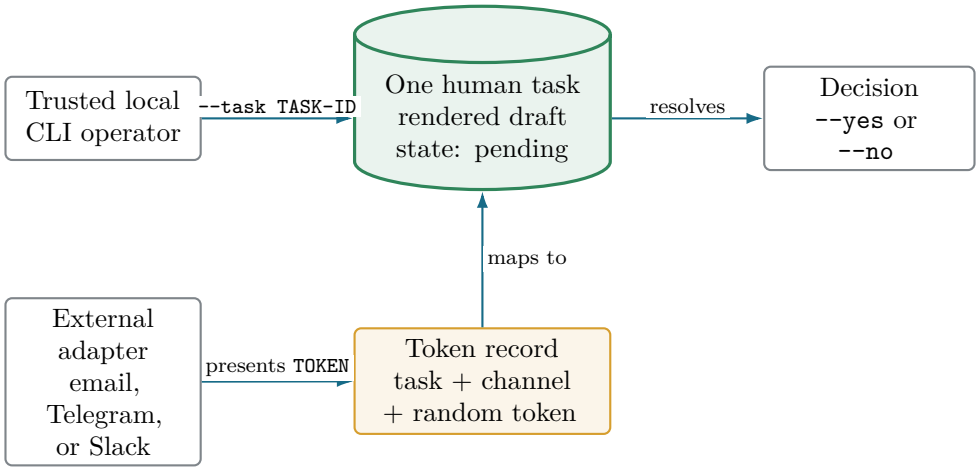

Figure 5: The human task is the pending work. A task ID is a local locator; a token is an adapter-facing credential that resolves to that same task. The separate `--yes`/`--no` flag supplies the decision.

22.2 Terminal A: start the workflow

From the project root:

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
STORE="$ZIPPERGEN_HOME/runs/manual-review.sqlite"

zippergen run zippergen/examples/tutorial_review.py:tutorial_review \
  --llm mock \
  --input 'request=Explain why the sky is blue in two sentences.' \
  --input 'max_retries=2' \
  --store "$STORE" \
```

```
  --timeout 0
```

Each `--input` occurrence supplies one `name=value` pair. The names match the parameters in `tutorial_review(request, max_retries)`. ZipperGen parses `max_retries=2` as an integer. For many inputs, the two flags may be replaced by one JSON object:

```
--input-json '{"request":"Explain why the sky is blue.","max_retries":2}'
```

`STORE` is merely a short shell variable chosen by this tutorial, not a special ZipperGen setting. The shell replaces `$STORE` with the full SQLite path before running each command. Terminal A prints that path and then waits. Leave it running. `--timeout 0` means no execution deadline; use this only when waiting is intentional.

22.3 Terminal B: inspect status and tasks

Open a second terminal, enter the repository, and repeat both variable assignments because a new terminal does not inherit them:

```
cd "$HOME/Projects/zippergen-tutorial"
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
STORE="$ZIPPERGEN_HOME/runs/manual-review.sqlite"

zippergen status \
  --store "$STORE"

zippergen tasks \
  --store "$STORE"
```

Status should say `waiting`. The tasks command prints the rendered draft and a task identifier. Copy the task identifier. Choose exactly one of the following decisions; once the task is decided, do not run the other command for the same task.

Approve it:

```
zippergen approve \
  --store "$STORE" \
  --task <TASK-ID> --yes
```

Or, to exercise the rejection branch instead, reject it explicitly:

```
zippergen approve \
  --store "$STORE" \
  --task <TASK-ID> --no
```

After rejection, Terminal A asks Writer for a revision and then waits on a *new* human task for the revised draft. List the pending tasks again, copy the new task ID, and make the next decision:

```
zippergen tasks --store "$STORE"

zippergen approve \
  --store "$STORE" \
  --task <NEW-TASK-ID> --yes
```

Every presented draft is a distinct task with its own task ID. Approving any draft finishes successfully. Rejecting the initial draft and both permitted revisions exhausts `max_retries=2`; Terminal A then returns the explicit failure notice and exits. Do not reuse an already-completed task ID.

The task ID is sufficient for this trusted local CLI exercise. External adapters such as email, Telegram, or Slack can instead use durable tokens. To display one, rerun the task listing as follows:

```
zippergen tasks \
  --store "$STORE" \
  --tokens --channel cli
```

Here `cli` is only a descriptive token-namespace label; it is not a workflow communication channel. Different adapter labels may have different tokens for the same task. A token replaces the task ID, while `--yes` or `--no` still determines the decision. For example, token-based rejection is:

```
zippergen approve \
  --store "$STORE" \
  --token <TOKEN> --no
```

After a draft is approved or the retry budget is exhausted, Terminal A prints a JSON result and exits. Confirm completion and view the trace:

```
zippergen status \
  --store "$STORE"

zippergen trace \
  --store "$STORE" --tail 30

zippergen tasks \
  --store "$STORE" --all
```

Why rerunning may finish immediately

The SQLite store records a completed result for this workflow. Running the same workflow against the same store replays committed work instead of repeating it. Point `STORE` at a new, explicitly named file for a genuinely new tutorial run, for example `STORE="$ZIPPERGEN_HOME/runs/manual-review-2.sqlite"`. Never delete a production store merely to force a rerun.

22.4 Recover after a terminal or computer crash

The foreground process stops, but the SQLite file remains. Open a new terminal, restore the working directory and tutorial variables, inspect the store, and rerun the same command with the same `STORE` value:

```
cd "$HOME/Projects/zippergen-tutorial"
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
STORE="$ZIPPERGEN_HOME/runs/manual-review.sqlite"

zippergen status \
  --store "$STORE"

zippergen run zippergen/examples/tutorial_review.py:tutorial_review \
  --llm mock \
  --input 'request=Explain why the sky is blue in two sentences.' \
  --input 'max_retries=2' \
  --store "$STORE" \
  --timeout 0
```

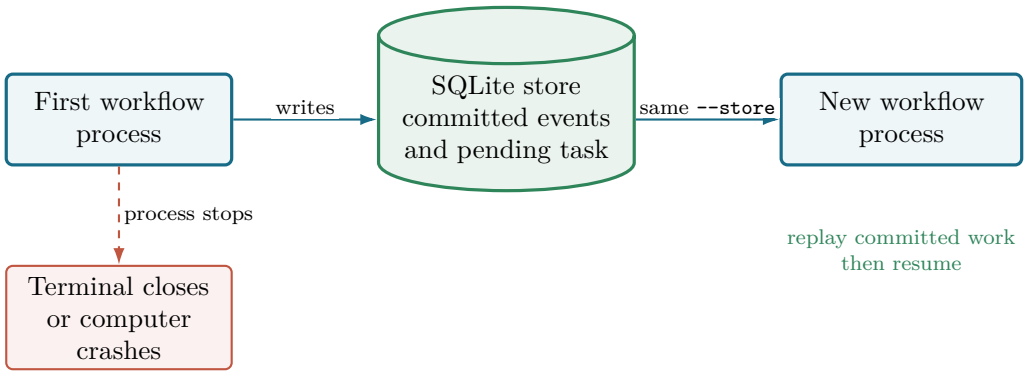

Figure 6: A crash removes the process, not the durable history. Starting a new process with the same SQLite store replays committed work and resumes.

The runner replays committed events and continues from the durable state. You do not recreate the checkout or virtual environment, reinstall packages, or activate the virtual environment: `uv run` selects it automatically. An in-memory run cannot be recovered this way because it has no SQLite record.

23 Develop and refine with a coding assistant

This section works on a copy so the canonical example remains available for comparison and deployment.

23.1 Create a branch and working copy

```
git switch -c tutorial/workflow-cycle
mkdir -p workbench
cp zippergen/examples/tutorial_review.py workbench/tutorial_review.py
git status --short
```

The `-c` form creates a new branch and therefore runs only once. If a branch of that name already exists from your own earlier attempt, use `git switch tutorial/workflow-cycle` instead. If `workbench/tutorial_review.py` already exists, inspect it before deciding whether to reuse it; do not overwrite work you do not understand.

Save a semantic baseline before any edit:

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
mkdir -p "$ZIPPERGEN_HOME/snapshots"
SNAPSHOT="$ZIPPERGEN_HOME/snapshots/tutorial-before-$(date +%Y%m%d-%H%M%S).json"

zippergen snapshot \
  workbench/tutorial_review.py:tutorial_review \
  -o "$SNAPSHOT"
echo "$SNAPSHOT"
```

The timestamp makes the baseline unique, so rerunning the exercise cannot silently replace the meaning you intended to preserve. Copy the absolute path printed by `echo`; it is the value used below.

23.2 Use a precise refinement prompt

In Studio, select the workbench copy and open the one automatically named pending refinement:

```
workflow select workbench/tutorial_review.py:tutorial_review
workflow refine
```

Paste the following text into the editor, save it, and exit:

```
# Add compliance review after exhaustion

If Reviewer exhausts the max_retries budget without approving a draft, a
separate Compliance human participant reviews the last draft. If Compliance
approves, release that draft to Requester; otherwise send the existing explicit
failure notice. Keep all Reviewer-approved paths and the retry bound unchanged.
```

Studio stores the text at `.zippergen/pending-refinement.md`, records its own semantic baseline, and generates the complete task. No prompt filename or baseline placeholder is required. Inspect and execute it:

```
workflow show pending
workflow status
workflow implement
```

The generated task supplies the nested skill, canonical specification, pending change, target, semantic baseline, assistant-check contract, and explicit no-deployment boundary. Studio starts Codex in the tutorial root with that fixed task.

23.3 Independently verify the assistant's work

Even if the assistant reports success, run these yourself. If this is a new terminal, first restore `SNAPSHOT` using the absolute path printed earlier:

```
cd "$HOME/Projects/zippergen-tutorial"
SNAPSHOT='<ABSOLUTE-SEMANTIC-BASELINE-PATH>'
```

Replace the placeholder and keep the single quotes. Then run:

```
git diff -- workbench tests
git status --short

zippergen validate workbench/tutorial_review.py:tutorial_review
zippergen show workbench/tutorial_review.py:tutorial_review \
  --detail protocol --communications
zippergen show workbench/tutorial_review.py:tutorial_review \
  --agent Reviewer
zippergen show workbench/tutorial_review.py:tutorial_review \
  --agent Compliance

zippergen diff \
  "$SNAPSHOT" \
  workbench/tutorial_review.py:tutorial_review
```

The diff should show a Compliance lifeline, a new human action, new messages and control structure on the retry-exhaustion path, and the deployment file-path change. It should not show changes to Reviewer-approved paths or to the retry bound. The semantic diff ignores comments and

formatting but detects workflow renames, action implementation changes, participant ownership, message/control context, execution order, parallel structure, and deployment requirements.

Run focused tests supplied by the assistant, then the full suite:

```
uv run --project zippergen python -m pytest -q
uvx pyright
```

Commit only after reviewing the change:

```
git add workbench/tutorial_review.py
git add tests/<ASSISTANT-CREATED-TEST-FILE>.py
git add specification.md
git diff --cached
git commit -m "Add compliance review tutorial variant"
```

Pushing is optional and depends on whether the branch belongs on your remote:

```
git push -u origin tutorial/workflow-cycle
```

23.4 Specification template for a workflow created from scratch

Enter `workflow create` and use this structure in the automatically opened `specification.md`:

```
# Workflow title

- Purpose and successful outcome: ...
- Participants and responsibilities: ...
- Inputs and initial owners: ...
- Messages and data crossing boundaries: ...
- LLM, deterministic, external-effect, and human actions: ...
- Decision and loop owners: ...
- Parallel work: ...
- Final outputs and owners: ...
- External services, credentials, and deployment constraints: ...
- Safety/idempotency requirements: ...
```

Later behavioral changes go into the single pending document opened by `workflow refine`. The assistant integrates that change into this clean canonical specification; after human review, Studio reconciles the pending document. Git, rather than a pile of active prompt fragments, preserves accepted history.

24 Create a guided named deployment

Return to the canonical example for a deterministic deployment exercise. Set the tutorial runtime directory in the current terminal:

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
```

Create the bundle, managed environment, profile, and service definitions, but do not start a background service yet:

```
zippergen deploy zippergen/examples/tutorial_review.py:tutorial_review \
  --name tutorial-review \
  --llm mock \
  --input 'request=Explain deployment in two simple sentences.' \
```

```
    --input 'max_retries=2' \
    --yes \
    --no-start
```

The default path performs these steps:

1. load the source workflow and deployment declaration;
2. collect defaults and command-line overrides;
3. copy declared source files into a timestamped bundle;
4. create a deployment-specific Python environment;
5. install ZipperGen and declared packages;
6. write the profile, run script, launchd plist, and systemd unit template;
7. run doctor checks; and
8. stop before service startup because of `--no-start`.

`--yes` accepts declared defaults non-interactively. Omit it to see the guided questions. It never means "approve arbitrary live effects"; it only controls configuration prompting.

24.1 Inspect the generated artifacts

```
find "$ZIPPERGEN_HOME" -maxdepth 5 -type f -print | sort
zippergen doctor tutorial-review
```

Before the first run, doctor normally warns that the SQLite store and log do not exist and that the service is not installed. Those warnings are expected; failures are not.

Path below `ZIPPERGEN_HOME`	Purpose
`deployments/tutorial-review.json`	Public deployment profile.
`deployments/tutorial-review.secrets.json`	Mode-0600 environment secrets, created when required. It is empty for the mock tutorial.
`deployments/tutorial-review.sh`	Stable launch script.
`deployments/io.zippergen.tutorial-review.plist`	Generated macOS launchd definition.
`deployments/zippergen-tutorial-review.service`	Generated Linux systemd user-unit template.
`environments/tutorial-review/`	Managed Python environment.
`apps/tutorial-review/<timestamp>/`	Timestamped source bundle for this deployment revision.
`runs/tutorial-review.sqlite`	Durable execution store.
`logs/tutorial-review.log`	Supervised-service standard output/error.

Managed environments are disposable build products, unlike source history and run stores. ZipperGen builds each replacement in a temporary sibling directory, prefers `uv` so that no `ensurepip` bootstrap is needed, installs dependencies there, and replaces the named environment only after success. A failed build leaves the previous environment unchanged and removes only its incomplete temporary directory.

25 Run the named deployment in the foreground

Foreground execution is the easiest way to test the deployment profile and bundle before involving a service manager.

25.1 Terminal A

```
cd "$HOME/Projects/zippergen-tutorial"
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
zippergen run-deployment tutorial-review
```

25.2 Terminal B

```
cd "$HOME/Projects/zippergen-tutorial"
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"

zippergen status tutorial-review
zippergen tasks \
  --store "$ZIPPERGEN_HOME/runs/tutorial-review.sqlite"

zippergen approve \
  --store "$ZIPPERGEN_HOME/runs/tutorial-review.sqlite" \
  --task <TASK-ID> --yes
```

Terminal A prints the result and exits. Then run:

```
zippergen status tutorial-review
zippergen trace tutorial-review --tail 30
zippergen doctor tutorial-review
```

This proves that the bundled code, managed Python, persisted inputs, mock LLM, SQLite store, and approval backend work together.

26 Optional: run as a supervised user service

The example is finite, while supervised deployments are primarily intended for long-running watchers and servers. To test supervision safely, use a second deployment name and stop it immediately after completion.

26.1 Prepare a fresh service deployment

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"

zippergen deploy zippergen/examples/tutorial_review.py:tutorial_review \
  --name tutorial-review-service \
  --llm mock \
  --input 'request=Demonstrate durable restart in two sentences.' \
  --input 'max_retries=2' \
  --yes --no-start

zippergen start tutorial-review-service --dry-run
```

Read the dry-run commands. On macOS they use `launchctl`; on Linux they use `systemctl --user`. If they look appropriate, start and enable the service:

```
zippergen start tutorial-review-service --enable
zippergen doctor tutorial-review-service
zippergen status tutorial-review-service
```

The status should become `waiting`. Test durability by restarting while the human task is still pending:

```
zippergen restart tutorial-review-service
zippergen status tutorial-review-service
```

The same task remains pending because it lives in SQLite, not only in process memory.

Because this service was started with `--enable`, its generated launcher records `ZIPPERGEN_HOME`. After a full reboot, the user service manager normally starts it again when the user session starts: after graphical login on macOS, or when the user systemd manager starts on Linux. No interactive shell export is needed by the service itself. You still repeat the `cd` and `export` commands before using ZipperGen CLI commands in a new terminal.

26.2 Watch logs and approve

In another terminal you may follow the log:

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
zippergen logs tutorial-review-service --follow
```

Press Control-C to stop following the log; that does not stop the service. List and approve the task:

```
zippergen tasks \
  --store "$ZIPPERGEN_HOME/runs/tutorial-review-service.sqlite"

zippergen approve \
  --store "$ZIPPERGEN_HOME/runs/tutorial-review-service.sqlite" \
  --task <TASK-ID> --yes

zippergen status tutorial-review-service
zippergen stop tutorial-review-service
```

Stop promptly after the finite workflow completes. The generated supervisor is configured to keep a deployment alive, which is correct for polling workflows such as call intake but unnecessary for this finite tutorial.

If the machine does not support launchd or systemd user services, keep using:

```
zippergen run-deployment tutorial-review-service
```

under your preferred terminal multiplexer or process supervisor.

27 Update, reconfigure, and redeploy

27.1 Change persistent configuration

For a long-running deployment, update declared fields and restart in one step:

```
zippergen configure tutorial-review-service \
  --input 'request=A new configured request.' \
  --input 'max_retries=3' \
  --yes --restart
```

For this finite example, the existing store already contains a completed result. Changing its input does not erase durable history or create a new job. Use a new deployment name/store for a new finite execution. For a real watcher, `configure --restart` is the normal pattern.

27.2 Redeploy changed source

After editing and validating the source, redeploy an existing named deployment:

```
zippergen validate zippergen/examples/tutorial_review.py:tutorial_review
zippergen deploy tutorial-review-service --yes
```

This creates a new timestamped source bundle and starts the named deployment. The stable SQLite store is retained, so already committed durable work is not repeated.

To deploy the assistant-modified workbench version as a separate test:

```
zippergen deploy workbench/tutorial_review.py:tutorial_review \
  --name tutorial-review-v2 \
  --llm mock \
  --input 'request=Test the compliance version.' \
  --input 'max_retries=2' \
  --yes --no-start
```

27.3 Switch lifelines to real models only after mock success

Replace ⟨*OPENAI-MODEL*⟩ and ⟨*ANTHROPIC-MODEL*⟩ with identifiers available to your accounts. Keep the inherited default on mock and override the two LLM-active lifelines. Omit `--yes` so the guided command asks for both enabled secrets without putting either in shell history:

```
zippergen configure tutorial-review-service \
  --llm mock \
  --llm-for 'Writer=openai:<OPENAI-MODEL>' \
  --llm-for 'Reviewer=claude:<ANTHROPIC-MODEL>' \
  --restart
```

The conditional OpenAI and Anthropic key fields now become active. ZipperGen stores the values in the deployment's private mode-0600 secrets file, loads them before importing the bundled workflow, and does not copy them into the public profile or service definition. Use `--llm-for Writer=inherit` to remove an existing override. If the service is intentionally stopped, omit `--restart`; test later with `run-deployment` or start it explicitly.

Never paste secrets into source or specifications

Do not commit API keys, include them in a coding-assistant prompt, pass them via `--set` on a shared shell history, or add them to a normal deployment profile. Use the guided secret prompt or a securely managed environment.

28 Troubleshooting

Managed environment creation aborts in `ensurepip`

Older deployments created the environment directly with Python's `venv.EnvBuilder(with_pip=True)`. Some uv-managed Python builds can abort inside the bundled `ensurepip` process, leaving a partially created `environments/NAME` and a raw `CalledProcessError/SIGABRT` traceback.

Update the editable ZipperGen installation, confirm `uv --version`, and repeat the same Studio `deploy` command. Current ZipperGen creates a fresh temporary environment with `uv`, installs into it, and atomically replaces the old or partial directory only after success. Do not manually delete the deployment profile, bundle, secrets, or run store. If creation still fails, Studio reports whether failure occurred during environment creation or dependency installation and confirms that the previous environment was retained.

`uv: command not found`

Restart the terminal after installing uv, then run `uv --version`. If it still fails, follow the PATH instructions printed by the official installer.

`studio` is reported as an invalid top-level command

The nested framework checkout or editable tool is older than this manual, or the terminal is in another project. From the tutorial project root, inspect before updating:

```
pwd
git -C zippergen status --short
git -C zippergen branch --show-current
git -C zippergen remote -v
uv sync --project zippergen --python 3.12
uv tool install --force --editable ./zippergen
zippergen --help
```

If the worktree is clean and this is the intended remote/branch, use `git -C zippergen pull --ff-only` and rerun the sync and editable tool installation. "Already up to date" only means Git found no newer commit on the configured upstream; it does not prove that the terminal is in the intended checkout or branch.

The shell reports `model` as an invalid command

This is expected if you entered `zippergen model` in the operating-system shell. `model` is a Studio command, not a top-level CLI subcommand. Start Studio from the Git root, then enter it at the Studio prompt:

```
zippergen
# The next line is entered only after zippergen [workflow]> appears:
model
```

If no workflow is selected, enter `workflow list` and then `workflow select`. Inspection and validation commands also open this selector automatically.

Studio says the specification is empty

Enter `workflow path` to see Studio's fixed path, then `workflow edit spec` to open it again. Add at least one real requirement, save, and leave the editor. The file must be valid nonempty UTF-8 text. No prompt directory, ID, or custom filename is required. If importing an existing document with the advanced `workflow create --file PATH` form, Studio reports the resolved absolute path and distinguishes a missing file, directory, empty file, and invalid UTF-8.

The shell says `command not found: --input` or `--store`

The preceding multiline command ended early. Re-enter the entire command and ensure every continuation backslash is the final character on its line. This same mistake can cause ZipperGen to report a missing workflow input because only the first part of the intended command actually ran.

`model` lists no Writer or Reviewer

Run `current` to confirm the selected workflow, then `workflow show actions`. Studio lists a lifeline in `model` only when that lifeline contains an actual `@llm` action. A pure, human, or message-only lifeline does not need a model. If the generated workflow was meant to contain those actions, treat their absence as a generation/validation problem rather than assigning a model to an unrelated participant.

A real model run reports an authentication or model error

Use `model` to inspect configurations and assignments, then use `model provider check NAME` for a fresh connection check and `model config check NAME` for exact model availability. Confirm that the account key belongs to that provider and that the named model is enabled for the account. Do not print the key while diagnosing. To continue the tutorial without network access or charges, enter `model default mock`, restore participant inheritance with `model inherit PARTICIPANT`, and start a fresh mock `run`.

The workflow file cannot be found

Check the current directory and spec:

```
pwd
ls zippergen/examples/tutorial_review.py
zippergen validate zippergen/examples/tutorial_review.py:tutorial_review
```

The text after the colon is the Python workflow object name, not the deployment name.

Git asks for `user.name` or `user.email`

Git cannot create the local commit until an author identity is configured. Follow your organization or hosting provider's identity policy. To configure only this tutorial checkout, run from its root:

```
git config user.name "Your Name"
git config user.email "your-address@example.com"
git commit -m "Add reviewed answer workflow"
```

Replace both example values. Repository-local configuration avoids changing other checkouts on the computer.

Validation or projection fails

Do not deploy. Read the first failing check, inspect the global view and the affected local projection, then ask the coding assistant to diagnose the error. Common causes are missing messages, a decision owner that does not possess the guard value, an output that is not present at the declared lifeline, or a parallel data dependency.

No human task appears

Confirm that Terminal A is still running and both commands use the same store:

```
zippergen status --store /exact/path/to/store.sqlite
zippergen tasks --store /exact/path/to/store.sqlite --all
```

If status is `done`, that store already contains a result. Use a new store path for a new finite run.

The service will not start

```
zippergen doctor tutorial-review-service
zippergen start tutorial-review-service --dry-run
zippergen logs tutorial-review-service --tail 100
```

On Linux, user systemd must be available. On macOS, `launchctl` must be available in the logged-in graphical user session. Use `run-deployment` as the foreground fallback.

Doctor shows warnings

Before first execution, the deployment store should already be ready and empty. A missing log or an uninstalled service before `start` is an expected warning. A `FAIL` result blocks safe startup and should be corrected.

A changed input does not create a new finite run

Durable stores intentionally replay prior committed work. Use a new deployment name or explicit new `--store` for a distinct finite job. Long-running workflows normally retain their store across restarts and redeployments.

The semantic diff shows an unexpected change

Use the full and local views to locate it. Do not dismiss unexpected lifeline, ownership, message, action-kind, control, protocol-order, or deployment changes as formatting. The semantic diff already ignores ordinary action formatting and comments.

29 Safe stopping, preservation, and cleanup

Inside Studio, one named deployment is one operational unit. The preferred cleanup path is:

```
deploy stop tutorial-review-service
deploy remove tutorial-review-service
```

`deploy stop` only stops supervision. It preserves every deployment artifact. `deploy remove` shows the exact ownership boundary, stops and unregisters the service if needed, and archives the whole deployment under `ZIPPERGEN_HOME/trash/deployments`. This includes its profile, private deployment secrets, launch files, immutable bundle revisions, managed Python environment, stable SQLite store and sidecars, and log.

The source checkout and project design remain. Named model configurations and connector configurations also remain because other deployments may use them. If another deployment refers to an allegedly owned path, Studio refuses removal rather than moving shared state.

Use the permanent form only when recovery is not required:

```
deploy remove tutorial-review-service --purge
```

The purge command requires the deployment name explicitly and asks you to type the exact name before deletion. Scripts may add `--yes` only after performing equivalent checks. There is deliberately no remove-all command. The foreground deployment needs no stop command after it exits, but its named profile and files still belong to the deployment and can be retired with the same `deploy remove` command.

To reset only this project's private Studio/development context while retaining the visible project, reopen Studio from the project root and enter:

```
project reset state
```

The preview separates preserved project files from affected private state. Answer `y` only after checking the project path and counts. On success, Studio prints the exact backup directory below `ZIPPERGEN_HOME/resets` and continues as `zippergen [no workflow]>`. Assistant requests, Studio command history, run records, SQLite development stores, private development keys, model/provider choices, and generated `.zippergen` task data are in that backup rather than being deleted. The canonical specification, workflow source, tests, manifest, Git repository, and framework checkout remain where they were. Deployment profiles, stores, logs, and running services are not part of this reset; retire them separately with `deploy remove`. The explicit `project reset state --yes` form is intended for scripts after equivalent checks, not casual interactive use. To restart the design tutorial instead, choose the fresh option from plain `project reset`; its preview shows that the manifest, specification, and legacy prompt material will also be archived while source, tests, and Git remain.

To abandon the workbench branch only after committing or otherwise preserving anything important, switch back normally:

```
git status --short
git switch main
```

Do not use destructive Git reset commands merely to clean up a tutorial.

30 End-to-end verification checklist

Record the observed result next to every item:

	Check	Result/date
☐	Studio opens the tutorial workflow without a `ZIPPERGEN_HOME` export or explicit store path.	
☐	`project` reports the workflow, specification, model and connector assignments, runs, and deployments. It also reports the project root and automatic `specification.md` path. The visible files contain no private runtime state.	
☐	`current`, `workflow list`, `workflow select`, and the selectable `workflow show` views make specification/refinement state, workflow name, participants, validation, model routes, provider connections, and runtime context clear.	
☐	`settings` reports global learning, interpreter, assistant, editor, and output preferences while language history remains project-local.	
☐	`editor set micro` remembers the installed global editor; `editor show`, `edit file`, and a one-off `--editor nano` behave as documented without MCP setup.	
☐	`workflow create` opens `specification.md` without asking for a filename, creates `.zippergen/current-task.md`, and prints a concise result table; `workflow status` exposes its lifecycle.	
☐	`workflow implement codex` launches Codex in the project root without a copied path or MCP setup; the assistant creates visible source plus a focused test at the requested paths.	
☐	After the assistant returns, Studio prints an implementation summary containing changed files, assistant checks, and the assistant report; `workflow status` reports `implemented`.	
☐	`workflow diff` shows the available specification and semantic baselines, while `workflow validate` independently checks the current workflow and every local projection.	
☐	`workflow refine` creates or reopens exactly one automatically named pending refinement; repeated edits do not create a pile of prompt files.	
☐	The assistant receives the canonical specification plus pending change. `workflow discard` archives and clears the pending request without reverting working-tree files.	
☐	A known stale implementation blocks deployment; a fresh clone with no local implementation record warns and proceeds after technical validation.	
☐	`model` lists connections, reusable configurations, and only LLM-active lifeline assignments; `model config check NAME` verifies a configuration without changing those assignments.	
☐	Studio `run` collects both inputs and presents rejection and approval in the same terminal.	
☐	Studio `resume` continues the same managed run after an interruption.	
☐	`model` distinguishes stored provider connections from timestamped local checks and model routing without displaying keys; secrets remain absent from source/request/run JSON and their private file has owner-only permissions.	
☐	`zippergen --help` works from the project root.	
☐	Direct mock run returns an approved and a rejected result.	
☐	`workflow validate` succeeds for all three projections.	
☐	Global, communications, selected-agent, and local views are clear.	

	Check	Result/date
☐	SQLite run reaches `waiting`; CLI approval reaches `done`.	
☐	A pre-edit semantic snapshot is created outside the source tree.	
☐	The assistant changes only the requested protocol behavior.	
☐	Semantic diff matches the intended change; tests and type checks pass.	
☐	Guided deploy creates a bundle, environment, profile, and doctor report.	
☐	A simulated environment-build failure retains the previous environment and reports a concise creation/install phase instead of a raw `ensurepip` traceback.	
☐	Named foreground deployment reaches a durable task and result.	
☐	Optional supervised restart preserves the pending task.	
☐	Logs, status, trace, doctor, restart, and stop commands are understood.	
☐	No secret appears in source, Git diff, public profile, or service file.	
☐	Plain `project reset` makes fresh-design, state-only, and cancel scopes explicit. The state-only choice preserves every project file; the fresh choice recoverably archives the manifest, specification, legacy prompts, and private state while preserving source, tests, Git, framework checkout, and deployment artifacts.	

If a step is difficult or surprising, that is product feedback. Save the exact command, output, operating system, and what you expected. The purpose of this exercise is not only to test your understanding; it is to test whether the development and deployment experience is genuinely self-explanatory.

A Compact command sheet

Recommended Studio path

```
cd "$HOME/Projects/zippergen-tutorial"
zippergen
```

Enter the following only after the `zippergen [workflow]>` prompt appears; do not prefix them with `zippergen`:

```
project
project rename "Reviewed Answer Tutorial" # optional logical rename
settings
settings set learning on
settings set assistant codex
settings set editor micro
settings set output banner
current
editor show
workflow create       # opens the automatic specification.md
workflow show spec
workflow edit spec
workflow path
workflow status
workflow implement codex # or: workflow implement claude
```

```
# After the coding assistant creates source:
workflow list         # show discovered entry points without validating
workflow import PATH.py[:WORKFLOW] # copy an existing workflow into this project
workflow select       # select one by number
workflow files        # list known implementation and resource files
workflow show source  # inspect authored Python before validation
workflow show
workflow validate
model default mock
model
model config check mock
run
resume
runs
run inspect
run tasks
run approve
run trace
workflow refine       # creates or reopens one pending change
workflow show pending
workflow implement codex # or: workflow implement claude
workflow diff         # specification and semantic before/after
workflow validate     # structural and projection check
workflow history
```

The listing above stays on the free mock backend. Only after following the account, billing, and key-safety instructions in the real-model readiness section, the optional paid-model commands are:

```
model provider configure openai
model config create drafting
model config check drafting
model assign Writer drafting
model provider configure anthropic
model config create careful-reviewer
model config check careful-reviewer
model assign Reviewer careful-reviewer
model
run
model default mock
model inherit Writer
model inherit Reviewer
```

Only after reading the deployment chapters and choosing mock/fake services, enter this prepare-first operational sequence:

```
model default mock
model inherit Writer
model inherit Reviewer
deploy tutorial-review-studio --no-start
deploy doctor
deploy show
deploy start
deploy show
deploy tasks
deploy logs
deploy restart
deploy show
deploy stop
deploy remove tutorial-review-studio # safe archive after the tutorial
```

```
exit
```

No `ZIPPERGEN_HOME` export or `STORE` variable is required for this path. Real model overrides are optional; use `model default mock` and `model inherit PARTICIPANT` to stay on the free mock path. If a coding assistant is unavailable, select `zippergen/examples/tutorial_review.py:tutorial_review` instead of the generated workflow.

Setup: new checkout

```
mkdir -p "$HOME/Projects"
cd "$HOME/Projects"
mkdir zippergen-tutorial
cd zippergen-tutorial
git init
git clone https://github.com/zippergen-io/zippergen.git zippergen
uv python install 3.12
uv sync --project zippergen --python 3.12
uv tool install --force --editable ./zippergen
zippergen --help
zippergen studio --project .
# Inside Studio:
project init zippergen-tutorial
```

Inspect and validate

```
zippergen validate zippergen/examples/tutorial_review.py:tutorial_review
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail
    overview
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail
    protocol
zippergen show zippergen/examples/tutorial_review.py:tutorial_review \
  --detail protocol --communications
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail
    actions
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --detail full
zippergen show zippergen/examples/tutorial_review.py:tutorial_review --agent
    Reviewer
zippergen show zippergen/examples/tutorial_review.py:tutorial_review \
  --agents Writer,Reviewer
```

Durable local execution

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
STORE="$ZIPPERGEN_HOME/runs/manual-review.sqlite"

zippergen run zippergen/examples/tutorial_review.py:tutorial_review \
  --llm mock \
  --input 'request=Explain the sky.' --input 'max_retries=2' \
  --store "$STORE" --timeout 0

zippergen status --store "$STORE"
zippergen tasks --store "$STORE"
zippergen approve --store "$STORE" \
```

```
  --task <TASK-ID> --yes
zippergen trace --store "$STORE"
```

Refinement safety gate

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
mkdir -p "$ZIPPERGEN_HOME/snapshots"
SNAPSHOT="$ZIPPERGEN_HOME/snapshots/workflow-before-$(date +%Y%m%d-%H%M%S).json"
zippergen snapshot path/to/workflow.py:workflow \
  -o "$SNAPSHOT"
# edit and test the workflow
zippergen validate path/to/workflow.py:workflow
zippergen diff "$SNAPSHOT" \
  path/to/workflow.py:workflow
```

Deployment and operation

```
export ZIPPERGEN_HOME="$HOME/Projects/zippergen-tutorial/tutorial-runtime"
zippergen deploy zippergen/examples/tutorial_review.py:tutorial_review \
  --name tutorial-review --llm mock \
  --input 'request=Explain deployment.' --input 'max_retries=2' \
  --yes --no-start
zippergen doctor tutorial-review
# Recommended first execution; approve it from a second terminal:
zippergen run-deployment tutorial-review
# Optional supervised-service exercise:
zippergen start tutorial-review --dry-run
zippergen start tutorial-review --enable
zippergen status tutorial-review
# Press Control-C when you have seen enough output; this does not stop service:
zippergen logs tutorial-review --follow
zippergen restart tutorial-review
zippergen configure tutorial-review --yes --restart
zippergen deploy tutorial-review --yes
zippergen stop tutorial-review
```

B Complete running-example source

The listing below is imported from the executable file at document build time. Both files are part of the same Git repository. The build runs from this `docs` directory, so the relative path is stable for both `make` and ordinary TeX editors.

Listing 1: examples/tutorial_review.py

```
1 # pyright: reportInvalidTypeForm=false, reportGeneralTypeIssues=false,
    reportOperatorIssue=false, reportCallIssue=false, reportAttributeAccessIssue=
    false, reportUnusedExpression=false, reportUnboundVariable=false,
    reportUndefinedVariable=false, reportReturnType=false
2 """Beginner tutorial: draft, assess, and ask a human to approve a reply.
3
4 Start with the built-in mock LLM and an in-terminal approval:
5
6     uv run python examples/tutorial_review.py
```

```
7
8  Use the durable SQLite runner and approve from another terminal:
9
10     uv run zippergen run examples/tutorial_review.py:tutorial_review \
11       --llm mock \
12       --input request="Explain why the sky is blue in two sentences." \
13       --input max_retries=2 \
14       --store /tmp/zippergen-tutorial-review.sqlite \
15       --timeout 0
16 """
17
18 from zippergen import (
19     DeploymentField,
20     DeploymentSpec,
21     Lifeline,
22     human,
23     llm,
24     pure,
25     workflow,
26 )
27
28
29 # Each lifeline is one sequential participant or responsibility boundary.
30 Requester = Lifeline("Requester")
31 Writer = Lifeline("Writer")
32 Reviewer = Lifeline("Reviewer")
33
34
35 @llm(
36     system="Write a clear, concise, and factual reply to the request.",
37     user="Request: {request}",
38     parse="text",
39     outputs=(("draft", str),),
40 )
41 def draft_reply(request: str) -> None: ...
42
43
44 @llm(
45     system=(
46         "Act as a careful reviewer. Identify factual, clarity, or safety "
47         "concerns in the proposed reply. Be concise."
48     ),
49     user="Draft to assess: {draft}",
50     parse="text",
51     outputs=(("concerns", str),),
52 )
53 def assess_draft(draft: str) -> None: ...
54
55
56 @human(
57     kind="confirm",
58     context="Draft:\n{draft}\n\nAutomated reviewer notes:\n{concerns}",
59     instruction="Approve this draft?",
60     outputs=["approved: bool"],
61     submit_label="Approve",
62     cancel_label="Revise",
63 )
64 def approve_reply(draft: str, concerns: str) -> None: ...
65
```

```
66
67  @llm(
68      system="Revise the draft after a human rejection. Make it clearer and safer.",
69      user="Original request: {request}\nRejected draft: {draft}",
70      parse="text",
71      outputs=(("draft", str),),
72  )
73  def revise_reply(request: str, draft: str) -> None: ...
74
75
76  @pure
77  def require_nonnegative(max_retries: int) -> int:
78      """Reject an invalid retry budget before the workflow starts reviewing."""
79      if max_retries < 0:
80          raise ValueError("max_retries must be zero or greater")
81      return max_retries
82
83
84  @pure
85  def begin_retries() -> int:
86      """The initial draft has not consumed a revision attempt."""
87      return 0
88
89
90  @pure
91  def increment_retries(retries: int) -> int:
92      return retries + 1
93
94
95  @pure
96  def review_exhausted(draft: str, max_retries: int) -> str:
97      return (
98          f"Review stopped after {max_retries} revision attempt(s) without "
99          f"approval. Last draft: {draft}"
100     )
101
102
103 @workflow
104 def tutorial_review(
105     request: str @ Requester,
106     max_retries: int @ Requester,
107 ) -> str:
108     Requester: max_retries = require_nonnegative(max_retries)
109     Requester(request) >> Writer(request)
110     Requester(max_retries) >> Reviewer(max_retries)
111     Writer: draft = draft_reply(request)
112     Writer(draft) >> Reviewer(draft)
113     with Reviewer:
114         concerns = assess_draft(draft)
115         retries = begin_retries()
116         approved = approve_reply(draft, concerns)
117
118     while (not approved and retries < max_retries) @ Reviewer:
119         Reviewer: retries = increment_retries(retries)
120         Reviewer(draft) >> Writer(draft)
121         Writer: draft = revise_reply(request, draft)
122         Writer(draft) >> Reviewer(draft)
123         with Reviewer:
124             concerns = assess_draft(draft)
```

```
            approved = approve_reply(draft, concerns)

    if approved @ Reviewer:
        Reviewer(draft) >> Requester(draft)
    else:
        Reviewer: failure = review_exhausted(draft, max_retries)
        Reviewer(failure) >> Requester(draft)
    return draft @ Requester


# This data-only declaration drives `zippergen deploy`. The OpenAI key is
# requested only after the LLM setting is changed from `mock` to `openai:...`.
zippergen_deployment = DeploymentSpec(
    name="tutorial-review",
    description=(
        "Draft a reply with an LLM and wait for a durable human approval. "
        "The default mock backend needs no API key."
    ),
    fields=(
        DeploymentField(
            "llm",
            "LLM provider and model",
            target="llm",
            default="mock",
            required=True,
        ),
        DeploymentField(
            "request",
            "Request to answer",
            target="input",
            default="Explain why the sky is blue in two sentences.",
            required=True,
        ),
        DeploymentField(
            "max_retries",
            "Maximum revisions after the initial draft is rejected",
            target="input",
            default=2,
            required=True,
            help="Use zero or a positive integer.",
        ),
        DeploymentField(
            "openai_api_key",
            "OpenAI API key",
            target="env",
            env="OPENAI_API_KEY",
            secret=True,
            required=True,
            when="llm",
            when_values=("openai*",),
        ),
        DeploymentField(
            "anthropic_api_key",
            "Anthropic API key",
            target="env",
            env="ANTHROPIC_API_KEY",
            secret=True,
            required=True,
            when="llm",
```

```
            when_values=("anthropic*", "claude*"),
        ),
    ),
    files=("examples/tutorial_review.py",),
)


if __name__ == "__main__":
    tutorial_review.configure(
        "mock",
        execution="memory",
        mock_delay=(0.0, 0.0),
    )
    answer = tutorial_review(
        request="Explain why the sky is blue in two sentences.",
        max_retries=2,
    )
    print(f"Result: {answer}")
```

C Build this document

The repository contains every document source dependency. From the nested ZipperGen Git root, with a TeX Live or MacTeX distribution installed, run:

```
make docs-manual
```

The PDF is then: `zippergen/docs/_build/workflow-development-deployment-guide.pdf`. TeX auxiliary files and the generated PDF live under the ignored `docs/_build` directory; the maintained source is the `.tex` file. Use `make docs` to build both this manual and the companion tutorial. Run `make docs-check` for an installation check without compiling. Detailed requirements and the equivalent direct `latexmk` command are in `docs/README.md`.